影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 17 章 邊緣 AI 與機器人視覺部署

本章學習目標

- 說明邊緣運算的動機，比較端上、近端邊緣、雲端三種 AI 辨識架構的取捨。
- 認識樹莓派 5 加 AI Camera 與 NVIDIA Jetson Orin Nano Super 兩大實作平台的定位與能力差異，並理解「感測器上推論」與「開放詞彙偵測」各自解決什麼問題。
- 運用量化、剪枝、ONNX、TensorRT 等技術最佳化模型以在邊緣裝置上執行。
- 量測與最佳化推論效能，理解延遲、吞吐量與功耗的權衡。
- 將視覺模組整合進 ROS 2 機器人系統，理解節點與話題的通訊架構。
- 完成一個端到端的機器人視覺部署，並評估系統的可靠度與失效模式。

叫獸開講|二十五美元的賭注：樹莓派如何讓「會看的電腦」變成每個人都買得起的東西

2006 年，劍橋大學聖約翰學院的電腦科學課程主任 Eben Upton 遇到一個他想不通的問題。每年來申請電腦科學的學生愈來愈少，而且來的人幾乎都沒寫過程式。這在他那個年代是不可思議的——1988 年，十歲的 Upton 用零用錢買了一台二手的 BBC Micro，自學 BASIC、自己寫遊戲;那個年代的家用電腦一開機就是程式提示符號，你想玩什麼，常常得自己先把它寫出來。二十年後，年輕人用電腦的時間比以前多得多，卻幾乎沒有人在「對電腦說話」。Upton 的診斷很具體：不是這一代比較笨，而是家裡那台電腦太貴、太重要了——它是爸媽報稅、哥哥姊姊交報告的機器，你怎麼敢拿它亂搞?

他的對策也同樣具體：做一台便宜到弄壞也不心疼的電腦。最早的原型在他的辦公桌上誕生，用的是一顆小小的微控制器，連接電視與鍵盤。接下來幾年，他與幾位同事一邊在 Broadcom 上班、一邊利用晚上與週末把它做出來，並成立了一個慈善基金會，把賣機器的利潤全部投回教育。2012 年 2 月 29 日，樹莓派正式開賣。Upton 原本希望能賣出一千台;他後來說，「一萬台對我們來說就已經是很棒的結果了」。結果開賣當天訂單就把兩家經銷商的網站打掛，到 2015 年初，銷量已超過五百萬台。

但真正和本課程有關的，是接下來發生的事。當一台能跑 Linux 的電腦只要二十五美元，它最先被拿去做的事情之一，就是「看」。2013 年，一位英國的業餘愛好者 Dave Akerman 用樹莓派控制一台相機，綁在高空氣球上送到三萬公尺的高空——連同基金會的吉祥物泰迪熊 Babbage 一起，在平流層拍下照片、再讓它跳傘回到地面。他說得很直白：樹莓派提供了一個便宜的方法，去控制一台畫質不錯的相機，並把影像與遙測資料傳回來。2015 年，兩台裝著相機的樹莓派被送上國際太空站;英國太空人 Tim Peake 在軌道上執行的，是中小學生自己寫的程式——有的量測艙內環境，有

的拍攝地球並分析影像。在那之前，「把影像處理程式送上太空」是國家級機構的事;在那之後，它變成一份作業。

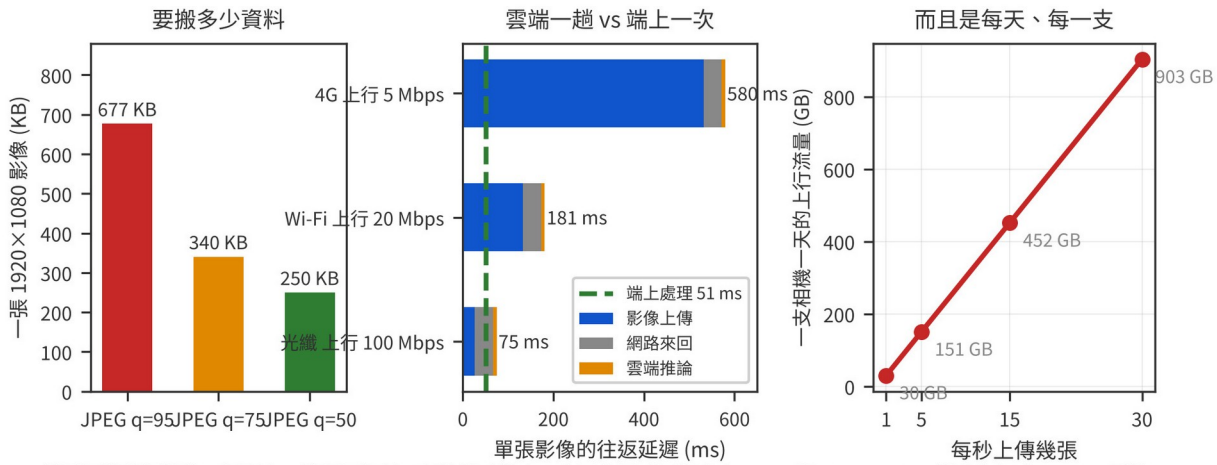
這條線一路延伸到 2024 年，而且拐了一個很有意思的彎。樹莓派推出 AI Camera，核心是 Sony 的 IMX500 智慧視覺感光元件——它把神經網路的推論做在感測器晶片裡。請注意這句話的份量：影像不必先傳給處理器，推論就已經完成了，主機拿到的是「畫面裡有什麼」，而不是「畫面」。從「便宜到能讓學生控制相機」，走到「便宜到讓相機自己會看」，中間隔了十二年。而本章要處理的，正是這十二年累積下來的工程問題：當你要把前面十六章的演算法，從有插電、有散熱、有充裕記憶體的筆電，搬到一顆靠電池、會發熱、記憶體以 MB 計的裝置上時，會發生什麼事。

思考題：Upton 認為年輕人不寫程式的原因，不是能力問題，而是「家裡那台電腦太貴重，沒人敢亂搞」。回想你自己學得最快的一次經驗，是不是也發生在一個「搞壞了也沒關係」的環境裡?這對你設計自己的學習方式——以及第 18 章的期末專題——有什麼啟發?

17.1 為什麼需要邊緣運算

17.1.1 從雲端回到現場

在深度學習興起的頭幾年，主流想法是「把運算放到雲端」：裝置只負責拍照，把影像傳到雲端伺服器，由強大的 GPU 完成辨識，再把結果傳回來。這個架構有明顯的優點——裝置可以很便宜、模型可以很大、更新也方便。但用在機器人與工業現場，它很快就撞牆了。

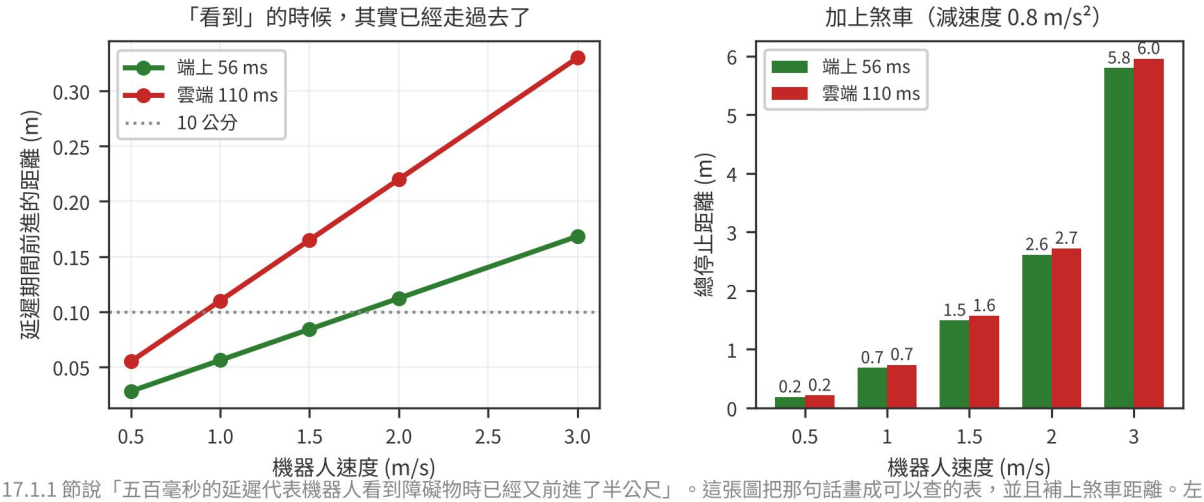


17.1.1 節列了邊緣運算的五個理由，這張圖把前三個換算成數字。左圖先量「要搬多少」：一張 1920×1080 的影像存成 JPEG，品質 95 時 677 KB、品質 75 時 340 KB、品質 50 時 250 KB。壓得更狠當然更省，但第 11 章量過壓縮會吃掉細節，而第 16 章的圖 16-13 又告訴我們細節正是小物件的命脈——這條路並不能無限壓下去。中圖是延遲的帳。以品質 75 的 340 KB 為例，在 4G 上行 5 Mbps 的條件下光是上傳就要 532 毫秒，加上約 40 毫秒的網路來回與 8 毫秒的雲端推論，一趟是 580 毫秒;就算換成 100 Mbps 的光纖，也還要 75 毫秒，因為那 40 毫秒的來回延遲與頻寬無關、壓不下去。綠色虛線是同一張影像在本機用 ONNX 跑 YOLO26n 的實測延遲 51 毫秒——注意它比最好的雲端情境還低，而且不會因為網路塞車而變成十倍。右圖是頻寬與成本：一支相機以 15 fps 持續上傳，一天就是 452 GB;一條產線三十支相機，一個月就是數十 TB。這三張圖合起來說明一件事：把運算搬回現場，買到的不只是速度，還有「不受網路狀況左右」這個性質。至於隱私與可靠性，它們無法畫成長條圖，但道理一樣直接——沒有上傳的影像，就不會外洩;不依賴網路的系統，斷網時仍然看得見。(本圖的頻寬與伺服器時間是常見的假設值，影像大小與端上延遲則是實測;換到你的場域請重新量一次。)

圖 17-1　雲端來回一趟，到底花掉什麼

邊緣運算(Edge Computing)的核心主張是：把運算搬回資料產生的現場。理由有五，每一個都攸關機器人視覺的成敗：

1. 延遲(Latency)：影像上傳、雲端運算、結果回傳，一來一回動輒數百毫秒。對一台以每秒一公尺行進的機器人，五百毫秒的延遲代表它「看到」障礙物時已經又前進了半公尺——這在避障、人機協作安全上是不可接受的。這裡請記住一個很好用的換算：延遲乘上速度，就是「資訊過期的距離」。圖 17-2 把這條關係畫了出來，而它還帶出一個比延遲更重要的事實——真正的停止距離是反應距離加上煞車距離，前者與速度成一次方、後者與速度成平方。實測以每秒一公尺、減速度 0.8 m/s² 計算：反應距離只有 0.15 公尺，煞車距離卻有 0.62 公尺。所以在低速時延遲是主角，到了高速則是煞車說話;而「把速度減半」往往是整個系統裡最便宜也最有效的旋鈕(推導 17.4.3 會把這條預算完整算一遍)。



17.1.1 節說「五百毫秒的延遲代表機器人看到障礙物時已經又前進了半公尺」。這張圖把那句話畫成可以查的表，並且補上煞車距離。左圖是純粹的感知延遲：以每秒一公尺行進時，端上處理的 56 毫秒等於 6 公分，而雲端來回的 110 毫秒等於 11 公分。速度拉到每秒三公尺（相當於快走到小跑），雲端架構光是在等結果就滑行了 33 公分。右圖加上物理：停下來還需要煞車距離，以 0.8 m/s² 的減速度估算，速度的平方才是主角。每秒一公尺時總停止距離是 0.68 公尺（端上）對 0.73 公尺（雲端）;到了每秒三公尺，兩者是 5.79 與 5.96 公尺。這張圖要傳達三件事。其一，延遲不是抽象的數字，它可以直接換算成公分，而公分是會撞到東西的單位。其二，感知延遲與速度成正比、煞車距離與速度平方成正比，所以高速時真正吃掉距離的是煞車——這表示一味壓低模型延遲會有報酬遞減，限制速度往往比換更快的模型有效得多。其三，反過來說，這張圖也給了你一個設計工具：先訂下「要在多遠之前停住」，再回推可容許的總延遲，這就是 17.4.2 節的延遲預算（圖 17-13 會把整條鏈算完）。

圖 17-2　延遲換算成距離：機器人在反應時間裡走了多遠

2. 頻寬與成本：一路 1080p 30fps 的影像串流，持續上傳的頻寬與雲端運算費用相當可觀;若是一整條產線數十支攝影機，成本更是驚人。
3. 可靠性：工廠、田間、隧道、災難現場往往網路不穩或根本沒有網路。一個「斷網就瞎掉」的機器人，是不可靠的機器人。
4. 隱私：影像是高度敏感的資料。承第 16 章的倫理討論，把人臉影像上傳雲端，與在裝置上處理完只回報「有幾個人」，對隱私的意涵天差地別。端上處理是最有力的隱私保護設計之一。
5. 功耗與自主性：電池供電的機器人與無人機，無法負擔持續高功率的無線傳輸;而自主系統的本質，就是不依賴外部連線也能運作。

17.1.2 三種 AI 辨識架構

這正是第 2 章就介紹過、並在第 10、16 章反覆呼應的三種架構，現在可以完整地整合起來：

表 17-1 三種 AI 辨識架構的比較

關鍵觀念是：這三者不是互斥的選擇，而是可以分層協作的。承第 10.7.3 節的門禁案例：端上用輕量的方式判斷「有沒有人臉」(高頻、低算力、保護隱私)，偵測到才把那一瞬間的影像交給後端做「是誰」的精細辨識(低頻、高算力)。這種「端上預篩、後端精算」的分層設計，兼顧了即時性、成本與隱私，是實務上最常見的架構模式。值得注意的是，這條分界線正在往前移動：17.2.1 節的 AI Camera 把第一層的推論直接做進感測器晶片裡，於是「端上預篩」不再需要一顆處理器來做——主機拿到的已經是結果，而不是影像。

17.2 兩大實作平台：樹莓派 5 與 Jetson Orin Nano Super

本書一路採用雙平台教學，現在正式比較它們的定位——這也是您在課程中希望學生建立的核心判斷力：選對工具，比用最強的工具更重要。這兩個平台剛好代表邊緣 AI 的兩種截然不同的思路：一個把推論推到最前端（感測器裡），另一個把算力集中在機器人身上;它們不是高階與低階的關係，而是兩種不同的系統設計。

17.2.1 樹莓派 5 與 AI Camera：把推論搬進感測器裡

承叫獸開講，樹莓派 5 是一台完整的 Linux 單板電腦（本章以 8 GB 記憶體的版本為例），能跑 Python、OpenCV、PyTorch 與本書前面所有的程式。真正特別的是搭配它的 AI Camera 模組。它的核心是 Sony IMX500 智慧視覺感光元件：一顆 1,230 萬畫素（4056×3040）的 CMOS 感測器，把神經網路的推論加速器做在同一顆晶片上。換句話說，你不需要外掛任何加速棒或加速卡，感測器自己就會跑模型。

它的運作方式值得仔細看一次，因為它和本書前面所有的做法都不同。傳統的流程是：感測器輸出影像 → 主機解碼與前處理 → 主機跑模型 → 得到結果。IMX500 的流程是：感測器輸出影像，同時附上一包「張量後設資料」（tensor metadata），裡面就是模型的輸出——偵測框、類別、分數。主機只要讀那包後設資料就好。libcamera、Picamera2 與 rpicam-apps 都已經支援這個欄位，所以在程式裡取用它並不複雜。兩條路徑的差別因此不只是快慢，而是「主機要不要參與」：傳統路徑上，解碼與推論這兩件事都得由主機親自做;感測器路徑上，主機只是讀一包很小的後設資料，處理器因此空下來，可以拿去做導航、控制或使用者介面。

這個設計買到的是三樣東西，而且都直接對應 17.1.1 節的五個理由。一是主機的時間：樹莓派的處理器可以拿去跑導航、控制或使用者介面，不必被視覺吃掉。二是頻寬與功耗：影像不必傳出去，連解碼都省了。三是隱私：這是最有力的一種隱私設計——不是承諾不看，而是影像根本沒有離開感測器（承第 16 章的倫理討論與 17.1.1 節）。

它的限制同樣明確，而且全部來自同一件事。感測器晶片上的記憶體額度很小，而且這個額度要同時容納韌體、網路權重與推論時的工作記憶體。由此衍生出三條硬

規定：輸入張量最大 640×640;輸入資料型別只收 8 位元整數（int8 或 uint8）——連 FP16 都不支援;能跑的模型清單很短，目前官方工具鏈支援的是 YOLOv8n 與 YOLO11n 這一級，本書第 16 章用的 YOLO26 還不在清單上。換句話說，INT8 在這裡不是一個可以權衡的選項，而是必要條件（這也讓 17.3.1 節與推導 17.3.4 的那筆帳變得具體）。

部署的流程因此多了一步「轉檔」。你在電腦上訓練好模型之後，要用官方的轉換工具鏈把它轉成感測器看得懂的格式，過程中會自動套用訓練後量化（PTQ，見 17.3.1 節與推導 17.3.4）。以 Ultralytics 為例，匯出與部署大致是這樣：

```
# 在電腦上：匯出成 IMX500 的部署格式（會自動做 INT8 量化）
from ultralytics import YOLO
model = YOLO("yolo11n.pt")           # 注意：目前支援到 YOLO11n 這一級
model.export(format="imx", data="coco8.yaml")   # 產生部署用的封裝檔

# 在樹莓派上：安裝韌體與工具
#   sudo apt update && sudo apt full-upgrade
#   sudo apt install imx500-all
#   sudo reboot
# 之後以 Picamera2 讀取影像與「張量後設資料」，
# 主機端不需要再跑一次模型。
```

最後是一個容易被誤解的地方，必須說清楚。感測器上推論並不比較快。感測器上的加速器是為低功耗與小面積設計的，不是為了極速;它跑同一個模型的速度，和一台普通電腦的處理器並沒有拉開差距。它買到的不是速度，是「主機的時間」與「不必搬影像」。所以它最適合的場景是：主機還有別的正事要做，或者主機大部分時間應該在睡覺（圖 17-4 的功耗帳）。若你的系統只有視覺這一件事、而且主機閒著，那麼它帶來的好處就小得多——這又是一次「適合勝於最強」。

17.2.2 Jetson Orin Nano Super：機器人的視覺大腦

Jetson 系列是 NVIDIA 專為邊緣 AI 設計的模組化電腦。本章以 Jetson Orin Nano Super 開發套件為例：它採用 Ampere 架構的 GPU，有 1024 個 CUDA 核心與 32 個張量核心，AI 效能約 67 TOPS，而功耗可以在 7 瓦到 25 瓦之間設定。它跑的是 Linux，支援 CUDA、TensorRT、PyTorch、ROS 2 等完整生態系——本質上是一台「能塞進機器人裡的小型 AI 工作站」。請特別注意那個可調的功耗範圍：它不只是省電的選項，更是一個設計旋鈕，讓同一塊板子能從「靠電池的無人機」一路用到「有插電的檢測站」（圖 17-4 會把它換算成續航時間）。

對機器人視覺而言，Jetson 的意義在於它讓第 15、16 章的深度學習模型能真正「上車」。YOLO 即時偵測、語意分割、姿態估計、視覺 SLAM——這些過去只能在桌機上跑的任務，現在能在機器人身上以合理的功耗即時執行。呼應第 15 章叫獸開講：當年 AlexNet 證明了 GPU 能訓練深度網路，而 Jetson 把同一套東西變成了機器人身上的一顆心臟。

這一級的算力解鎖了一類新的模型，值得在這裡特別介紹，因為它改變的不只是速度，而是「你能問什麼問題」。本書到第 16 章為止的偵測器，類別都是在訓練時就固定的：COCO 的那八十類，多一類都不行，要加就得重新標註與訓練。開放詞彙偵測（open-vocabulary detection）把這件事翻轉過來——類別變成一段文字，執行

時才決定。它的作法是把影像與文字投影到同一個向量空間，比對兩者的相似度;這類模型的代表是 OWL-ViT，而 NVIDIA 的 NanoOWL 專案把它用 TensorRT 最佳化，讓它能在 Jetson Orin 上即時執行。

NanoOWL 還提供一個有趣的「樹狀提示」語法，可以把偵測與分類巢狀地組合起來。例如寫 [a face [a nose, an eye, a mouth]]，它會先找出每一張臉，再在每一張臉的區域裡找鼻子、眼睛與嘴巴;而寫 (indoors, outdoors) 則是把整張影像分類成室內或室外。對機器人來說，這代表你可以在現場用一句話改變它要找的東西，而不必回實驗室重訓一輪。圖 17-3 以本機可執行的開放詞彙模型示範了這件事——同一張照片，只是換了幾個字，找到的東西就完全不同。



這張圖是實測，示範 17.2.2 節說的開放詞彙偵測，三張都是同一個模型、同一組權重，差別只有你輸入的那幾個字。左圖問 bus 與 person，找到 6 個——這是任何偵測器都會做的事。中圖換一張室內照片，問 lamp 與 shelf，找到 4 個。請注意這兩個詞都不在第 16 章那份八十類清單裡——換成 YOLO，你必須重新標註、重新訓練才能得到這個結果，而這裡只花了打字的時間。這就是開放詞彙的價值：類別不再是訓練時焊死的，而是執行時才決定的參數。作法上，模型把影像與文字投影到同一個向量空間，比對兩者的相似度，相似就算命中。NanoOWL 正是這一類模型的代表：以 OWL-ViT 為基礎，用 TensorRT 最佳化後在 Jetson Orin 上即時執行，還支援巢狀的樹狀提示——先找出每一張臉，再在每張臉裡找眼睛與嘴巴。對機器人而言這是個不小的改變：你可以在現場用一句話告訴它今天要找什麼。但右圖立刻澆了一盆冷水。同一張照片、同樣兩個概念，只是把詞改成複數的 lamps 與 shelves，結果是 0 個。模型比對的是那串字元產生的向量，不是你心裡的那個概念;它沒有「複數等於同一件事」這種常識。下一張圖會把這件事量成數字，連同開放詞彙的其他兩筆代價一起算。（本圖以 YOLOE 在本機實測。NanoOWL 需要 Jetson 與 TensorRT，無法在撰稿機器上執行，因此本書只介紹它的功能與用法，不列任何效能數字。）

圖 17-3　開放詞彙偵測：換一句話，就換一組類別

```
# NanoOWL 的典型用法（需在 Jetson 上，並先建好 TensorRT 引擎）
from nanoowl.owl_predictor import OwlPredictor

predictor = OwlPredictor(
    "google/owlvit-base-patch32",
    image_encoder_engine="data/owl_image_encoder_patch32.engine")

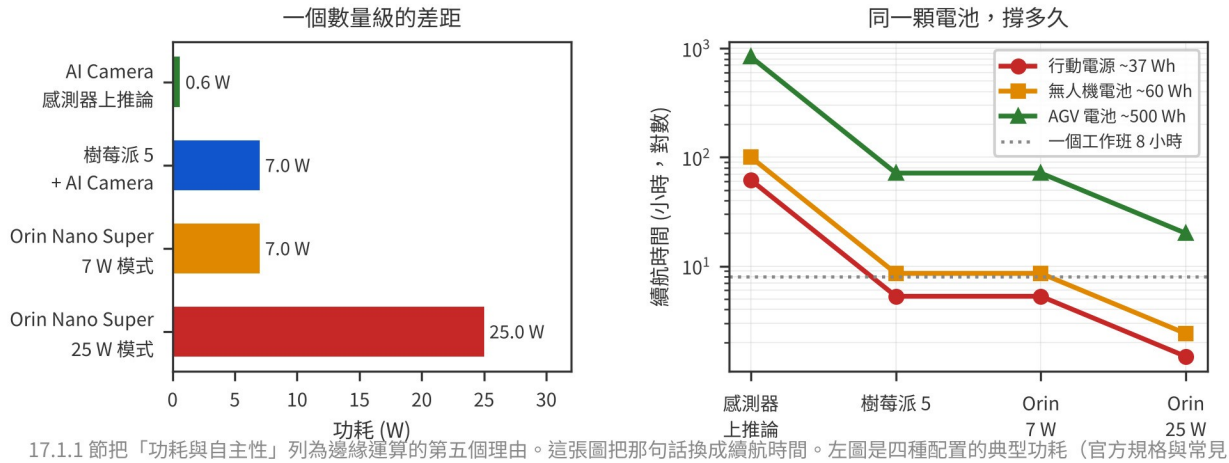
output = predictor.predict(image=image,
                           text=["a person", "a helmet"],
                           threshold=0.1)

# 樹狀提示（先找臉，再在臉裡找五官）：
#   --prompt="[a face [a nose, an eye, a mouth]]"
```

但開放詞彙不是免費的，而且代價比多數人想的大。它有三筆代價。第一筆最要命：它比對的是文字產生的向量，不是你心裡的概念——單數與複數、加不加冠詞、換一個同義詞，結果都可能天差地別（圖 17-3 的右圖就是一個例子）。這代表提示詞

本身就是一個必須在開發集上調的參數。第二筆是速度：它比同級的封閉詞彙模型重，因為每次推論都要同時處理影像與文字兩邊。第三筆是精度：它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。所以它的正確用法是：在專題前期用它快速試出「這個場景裡到底要找什麼」，等類別定下來之後，再訓練一個封閉詞彙的模型來真正上線。

選擇建議：若你的任務是固定的、類別已知、而且希望主機盡量閒著（環境監測門禁、產線上的單一檢項），樹莓派 5 加 AI Camera 是很划算的選擇——它便宜、省電、影像不出裝置。若你需要即時的深度學習推論、SLAM、多感測器融合，或者需要開放詞彙這類較重的模型，那就得用 Jetson Orin Nano Super 這一級的算力。兩者也常搭配使用：多個 AI Camera 節點分佈在環境中做第一層過濾，Jetson 作為集中處理與決策的大腦——這正是 17.1.2 節三層架構的實體。



17.1.1 節把「功耗與自主性」列為邊緣運算的第五個理由。這張圖把那句話換成續航時間。左圖是四種配置的典型功耗（官方規格與常見實測的估算值，非本機實測——這類數字必須在你的硬體上用電流錶量）。請注意第一列與第二列的關係：AI Camera 把推論做在感測器裡，模組本身約零點幾瓦，但它要掛在一台樹莓派 5 上，而樹莓派 5 光是開機待命就要好幾瓦——所以「感測器上推論很省電」這句話只有在「主機可以大部分時間睡覺」時才兌現。這正是它與 Orin Nano Super 最大的差別：Orin 的 7 W 到 25 W 是「正在全力推論」的功耗，而 AI Camera 的省電來自「主機不必醒著」。右圖是後果。一顆 37 Wh 的行動電源，帶樹莓派 5 加 AI Camera 約 5.3 小時，帶 Orin Nano Super 在 25 W 模式只剩 1.5 小時;一顆 500 Wh 的 AGV 電池帶 25 W 的 Orin 可以跑 20 小時，但那顆電池本身就有數公斤重。這張圖要說的是：功耗不是技術細節，它直接決定機器的形狀。想做一個靠太陽能掛在果園裡一整季的監測節點，那就必須讓主機大部分時間休眠、只在感測器報告「有東西」時才醒來——而這正是感測器上推論真正的用途（17.2.1 節）。想在無人機上跑開放詞彙偵測，25 W 會明顯吃掉飛行時間，所以實務上常見的折衷是「只在特定航點才啟動推論」。這些都不是軟體問題，是能量守恆。最後補一句與第 16 章呼應的話：省電最有效的手段往往不是把模型調快，而是「不要一直算」——降低處理影格率（圖 17-14）或讓感測器替你過濾，省下的是整段推論的電，而不是其中的幾個百分點。

圖 17-4　功耗與續航：自主性的算術

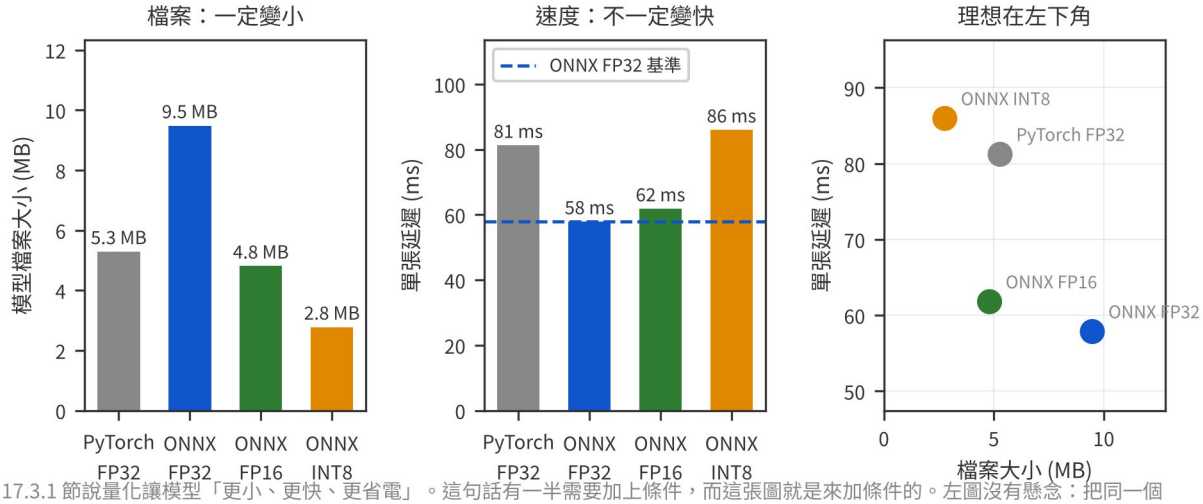
17.3 模型最佳化：讓大模型跑在小裝置上

直接把訓練好的模型丟到邊緣裝置，往往跑不動或跑太慢。模型最佳化是邊緣部署的核心技術，目標是在可接受的精度損失下，大幅降低運算量、記憶體與功耗。

17.3.1 量化

量化(Quantization)是最有效的手段：訓練時模型的權重與運算通常用 32 位元浮點數(FP32)，但推論時未必需要這麼高的精度。把它降到 16 位元浮點(FP16)或 8 位元整數(INT8)，模型大小可縮為二分之一到四分之一，而精度往往只損失一點點。不過「速度」這一項必須說得更小心，因為它不一定兌現。量化之所以可能加速，靠的是

兩條路徑：一是權重少搬一半，若這一層原本被記憶體速度卡住，搬得快就真的算得快;二是硬體有原生的低精度乘加指令(Jetson 的 Tensor Core 對 FP16、許多 NPU 與 DSP 對 INT8)，同一個週期能做更多次乘加。這兩條都要硬體配合，而第二條更挑。圖 17-5 在一顆沒有這些單元的 CPU 上實測：ONNX FP32 是 9.48 MB、33.5 毫秒 FP16 縮到 4.81 MB 但變成 42.6 毫秒，INT8 縮到 2.77 MB 卻慢到 58.7 毫秒——體積一路變小，速度一路變慢。原因是處理器沒有半精度單元時必須把 FP16 轉回 FP32 才能算，而動態 INT8 還要在執行期間統計激活值範圍、並在整數與浮點之間來回轉換，這些額外動作吃掉了省下來的乘法時間。精度那一面則是 FP16 幾乎免費、INT8 要付代價：圖 17-6 實測 mAP@0.5 從 61.61% 降到 59.19%。所以量化該拆成三個獨立的問題來看——裝不裝得下(一定有幫助)、掉多少精度(可以靠靜態量化或量化感知訓練救回)、會不會變快(只有在目標硬體上量過才知道)，推導 17.3.4 會把這三件事的算式各寫一遍。這個「用精度換效率」的思想，你應該覺得眼熟——它正是第 3 章量化(把連續的光強度離散化)與第 11 章 JPEG 量化(把 DCT 係數粗略化)的同一個核心概念，在模型層次的再現。本書反覆出現的觀念，再一次以新的面貌登場。

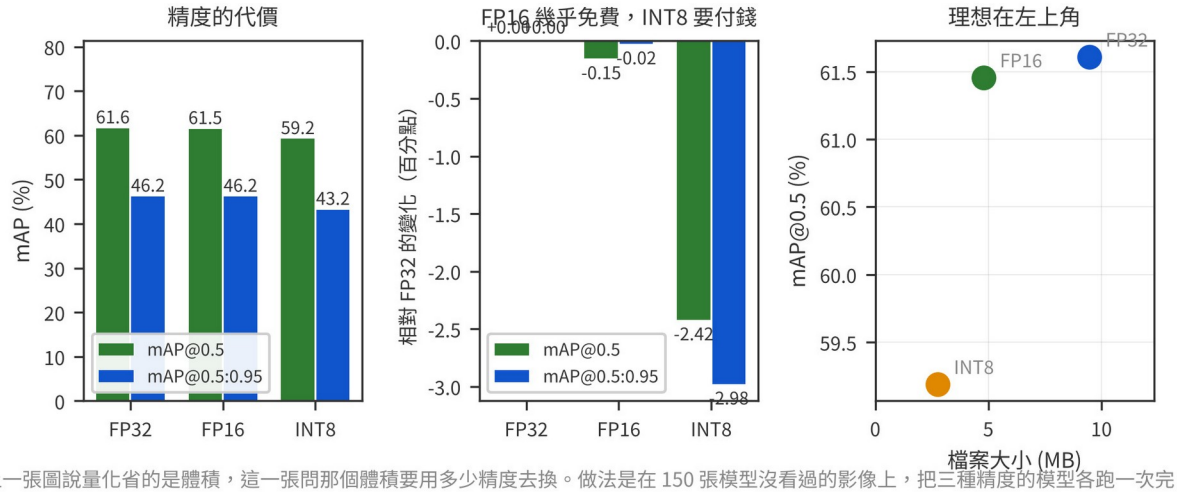


17.3.1 節說量化讓模型「更小、更快、更省電」。這句話有一半需要加上條件，而這張圖就是來加條件的。左圖沒有懸念：把同一個 YOLO26n 從 FP32 降到 FP16 再降到 INT8，檔案從 9.5 MB 變成 4.8 MB 再變成 2.8 MB，大約就是位元數的比例——這部分與第 3 章的影像量化、第 11 章的 JPEG 量化是同一個算術。中圖才是重點。在這台純 CPU 的機器上，FP32 的 ONNX 跑 58 毫秒，FP16 反而變成 62 毫秒，INT8 更慢到 86 毫秒——比沒量化還慢。原因不難理解：這顆 CPU 沒有原生的 FP16 運算單元，推論引擎得先把半精度轉回單精度才算得動，轉換本身就是成本;而動態 INT8 量化要在執行時即時決定縮放係數、並在整數與浮點之間來回搬運，省下的乘法時間被這些額外動作吃光。所以量化真正換到的是什麼?是「體積」與「記憶體頻寬」——這兩項在小裝置上往往才是瓶頸，而且它決定了模型裝不裝得進去。至於速度，要等硬體原生支援那個精度才兌現：Jetson 的 Tensor Core 對 FP16、微控制器與 NPU 對 INT8，都有專門的運算單元，那裡的加速是實打實的。這帶出本章最該記住的工程紀律：任何最佳化都必須在「目標硬體」上量過才算數。在筆電上量到的結論，換到 Jetson 或 IMX500 這類感測器可能完全相反。

圖 17-5　量化：檔案一定變小，速度不一定變快

17.3.2 剪枝與知識蒸餾

剪枝(Pruning)把網路中「貢獻很小」的權重或整個通道移除，讓模型變稀疏、變小。知識蒸餾(Knowledge Distillation)則訓練一個小模型(學生)去模仿大模型(老師)的輸出，讓小模型繼承大模型的能力。兩者都是「把模型變小但盡量保住能力」的策略，常與量化搭配使用。

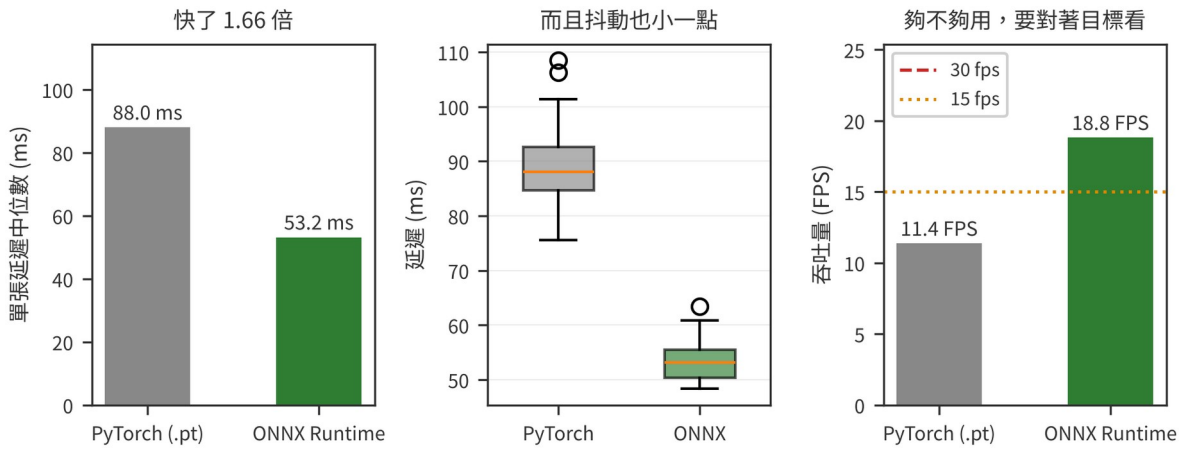


上一張圖說量化省的是體積，這一張問那個體積要用多少精度去換。做法是在 150 張模型沒看過的影像上，把三種精度的模型各跑一次完整評估。FP16 的結果幾乎可以說是免費的：mAP@0.5 從 61.61% 變成 61.45%，mAP@0.5:0.95 從 46.19% 變成 46.17%，兩項都在小數點後第二位打轉，而檔案少了一半。這很合理——半精度浮點仍有十位元的有效位數，而神經網路的權重本來就分布在一個不大的範圍內。INT8 就不一樣了：mAP@0.5 掉到 59.19%（少 2.42 個百分點）、mAP@0.5:0.95 掉到 43.22%（少 2.98 個百分點），換來的是檔案只剩 2.8 MB。要不要付這筆錢，取決於應用。八位元整數只有 256 個刻度，要把一層的權重與激活值都塞進這 256 格，靠的是為每一層挑一個縮放係數——挑得好不好，直接決定損失多少。本圖用的是最省事的「動態量化」，只量化權重;實務上若改用「訓練後靜態量化」，拿一小批代表性影像先跑一遍、統計每一層激活值的範圍再定縮放，精度通常可以救回大半。更講究的還有「量化感知訓練」，在訓練時就模擬量化誤差。所以 INT8 掉三個百分點不是定論，是「不花力氣時」的結果。最後提醒一句方法論：這些數字必須在模型沒看過的資料上量（第 16 章圖 16-18），否則量化的損失會被記憶效應蓋過去。

圖 17-6　量化的精度代價

17.3.3 推論引擎：ONNX 與 TensorRT

訓練用的框架(PyTorch、TensorFlow)為了靈活性，推論效率並非最佳。部署時通常會轉換成專用的推論格式：ONNX 是一個開放的中介格式，讓模型能在不同框架與硬體間流通(第 16 章程式 16-4 的 export 就是在做這件事);TensorRT 則是 NVIDIA 針對自家 GPU 的推論最佳化引擎，它會分析模型結構、融合運算層、選擇最佳核心、並套用量化，在 Jetson 上通常能帶來數倍的加速。對更受限的微控制器，則有 TensorFlow Lite for Microcontrollers 等專用的輕量執行環境;而 17.2.1 節的 IMX500 走的是另一條路——它有自己的轉換工具鏈，把模型編成感測器韌體看得懂的格式。值得特別強調的是：單純換成推論引擎，是本章最划算的一招。圖 17-7 實測同一顆 YOLO26n、同一張影像、同一台機器：PyTorch 的 .pt 中位 55.11 毫秒，ONNX Runtime 中位 32.82 毫秒，快了約 1.68 倍——而且精度一模一樣(圖 17-6)。加速來自三件事：把卷積與批次正規化這類推論時固定的線性變換事先融合成單一運算、拿掉自動微分為反向傳播保留的全部紀錄、以及把模型編成靜態圖而不必每一層都回到 Python 問下一步做什麼。代價是失去靈活性——靜態圖不能隨意改結構、除錯也麻煩;但部署時模型已經定案，這個代價等於不存在。換句話說，在開始調解析度、換小模型、或嘗試量化之前，先把模型匯出成部署格式量一次，因為這一步不花精度就能拿到的加速，往後幾步都未必拿得到。



17.3.3 節說訓練框架「為了靈活性，推論效率並非最佳」，部署時應轉成專用格式。這張圖量那句話值多少。做法是把同一個 YOLO26n 匯出成 ONNX，用 ONNX Runtime 在同一台機器、同一張影像上跑，與原本的 PyTorch 版本比較。結果是延遲中位數從 88.0 毫秒降到 53.2 毫秒，快了 1.66 倍;換算成吞吐量是 11.4 FPS 變成 18.8 FPS。注意這裡什麼都沒犧牲——模型是同一個、精度一模一樣（圖 17-6 會驗證這件事），純粹是因為推論引擎可以做訓練框架做不到的最佳化：運算元融合、常數摺疊、記憶體重用、針對這顆 CPU 選最快的核心實作。中圖另外顯示一件容易被忽略的好處：ONNX 版本的延遲分布也比較集中，對即時系統而言，抖動小本身就是價值（圖 17-12 會細談）。一個實務建議：匯出格式應該是部署流程的第一步，而不是最後的微調。它幾乎沒有風險（精度不變）、幾乎沒有成本（一行程式），而且是後續所有最佳化的基礎——量化、TensorRT、LiteRT，都是從這個中介格式往下走的。右圖則提醒：效能數字必須對著目標看。18.8 FPS 聽起來不錯，但若任務要 30 fps 就還差得遠;這時才輪到圖 17-5 與圖 17-8 的那些手段上場。

圖17-7　換一個推論引擎，就快了一截

架構層面的選擇同樣重要：與其把大模型硬壓小，不如一開始就選用為邊緣設計的架構——第 15 章的 MobileNet、第 16 章 YOLO 的 nano 版本，都是為此而生。這呼應了第 9、10、13 章反覆強調的判準：適合勝於最強。

程式 17-1 模型匯出與效能量測

```
from ultralytics import YOLO
import time, cv2

model = YOLO("yolo26n.pt")          # nano:為邊緣裝置設計

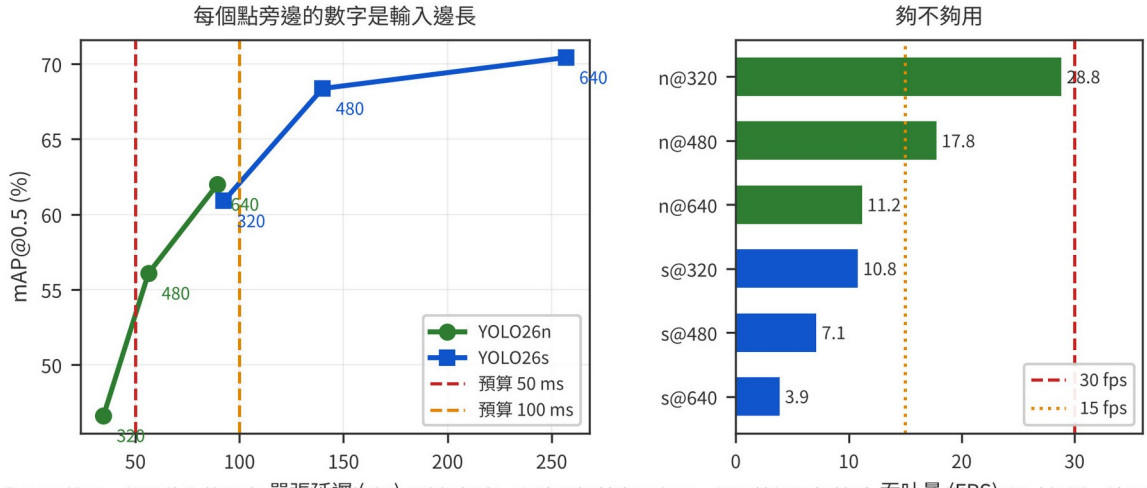
# --- 匯出為部署格式 ---
model.export(format="onnx")                     # 通用中介格式
# model.export(format="engine", half=True)      # TensorRT + FP16(Jetson)
# model.export(format="tflite", int8=True)      # TFLite + INT8(行動/微控)

# --- 量測推論效能(部署前必做) ---
img = cv2.imread("test.jpg")
model(img)                              # 暖機一次(第一次含初始化)

N = 50
t0 = time.time()
for _ in range(N):
    model(img, verbose=False)
dt = (time.time() - t0) / N
print(f"平均延遲:{dt*1000:.1f} ms,吞吐量:{1/dt:.1f} FPS")

# 效能不足時的調整順序:
#  1. 換更小的模型(n < s < m < l < x)
#  2. 降低輸入解析度(imgsz=640 → 416 → 320)
```

```
#  3. 量化(FP16 / INT8)
#  4. 用 TensorRT 等推論引擎
#  5. 降低處理影格率(不必每一影格都偵測)
```



程式 17-1 給了一份最佳化的順序：先換小模型，再降解析度。但這兩招其實是在同一個預算裡互相競爭，哪招划算要量過才知道。這張圖把兩個模型 × 三種輸入邊長共六種組合全部量一遍（延遲在本機實測，mAP 在 150 張保留影像上實測），畫成一條「延遲對準確度」的取捨曲線。關鍵的對照在中段。「大模型配小輸入」的 YOLO26s@320 是 92 毫秒、60.9%；「小模型配大輸入」的 YOLO26n@640 是 89 毫秒、62.0%。兩者差 3 毫秒、1.1 個百分點——幾乎疊在同一個點上。也就是說，在這個預算帶裡，程式 17-1 建議的那兩招（換小模型、降解析度）最後到達的位置差不多，沒有哪一招天生比較划算。往兩端走才有明顯的差異：預算壓到 20 毫秒以內時只剩 YOLO26n@320 可選（46.6%），而預算放寬到 90 毫秒以上時，YOLO26s@480 的 68.4% 把所有 n 系列的組合都甩開。所以沒有一條通用的規則，只有一條你自己量出來的曲線。這也是為什麼程式 17-1 給的是「嘗試的順序」而不是「正確的答案」。實務上的做法是：把這張圖在目標硬體上重畫一次，在圖上拉一條垂直線代表你的延遲預算，取這條線左邊最高的那個點。順帶一提，左圖也解釋了為什麼第 16 章圖 16-12 的解析度曲線會有倒 U：輸入放大到超過訓練尺度之後，準確度不升反降，而延遲還是照樣增加——那是整張圖上最不划算的區域。

圖 17-8　同一個延遲預算，該換小模型還是降解析度

17.3.4 推導：量化在算什麼，以及它為什麼不一定變快

17.3.1 節說量化讓模型「更小、更快、更省電」。這一小節把那句話拆成三件事來算，因為它們兌現的條件並不相同——圖 17-5 的實測就出現了「檔案變小、速度反而變慢」的情況，而理解下面這幾行算術，就知道那不是意外。

先看量化在做什麼。訓練好的權重是一堆 32 位元浮點數，它們實際的分布範圍通常不大。量化的基本做法是為一組數值（可能是一整層、也可能是一個通道）找出最大與最小值，把這個範圍線性地映射到整數刻度上。以 8 位元無號整數為例，可用的刻度只有 0 到 255 共 256 格：

```
縮放係數 s = (最大值 − 最小值) / 255
零點     z = round(−最小值 / s)
量化     q = clip(round(x / s) + z, 0, 255)
還原     x̂ = s × (q − z)
```

還原回來的 $\hat{x}$ 與原本的 x 之間會差一點，這個誤差最大是半格，也就是 $s/2$。這裡立刻看出兩件事。其一，誤差與「範圍寬度」成正比：若某一層裡剛好有一個離群的大權重，整個範圍被它撐開，其餘所有權重的精度就一起變差。這正是實務上要逐通道（per-channel）而非逐層決定縮放係數的原因，也是「剪掉離群值」這類校正手段有效的原因。其二，這個做法與第 3 章把連續亮度切成 256 階、第 11 章 JPEG 用量化表除掉高頻係數，是同一個思想：用精度換效率，而且換掉的那部分再也回不來。

接著算「省了多少」。這部分最單純，而且一定兌現：

```
權重所需位元組 = 參數量 × 每個參數的位元組數
  FP32: 4 bytes    FP16: 2 bytes    INT8: 1 byte
```

YOLO26n 有約 257 萬個參數，以 FP32 存放約 9.8 MB、FP16 約 4.9 MB、INT8 約 2.5 MB（圖 17-4 的實測）。體積決定了模型裝不裝得進去，而在微控制器這一級，裝不裝得下往往才是唯一的問題。

最後是「快了多少」，也就是最容易說錯的那一部分。量化之所以可能加速，有兩條完全不同的路徑。第一條是記憶體頻寬：權重少搬一半，若這一層原本是被記憶體速度卡住（memory-bound），那麼搬得快就真的算得快。第二條是運算單元：若硬體有原生的低精度乘加指令（Jetson 的 Tensor Core 對 FP16、許多 NPU 與 DSP 對 INT8），那麼同樣一個週期可以做更多次乘加，速度才會成倍數地提升。

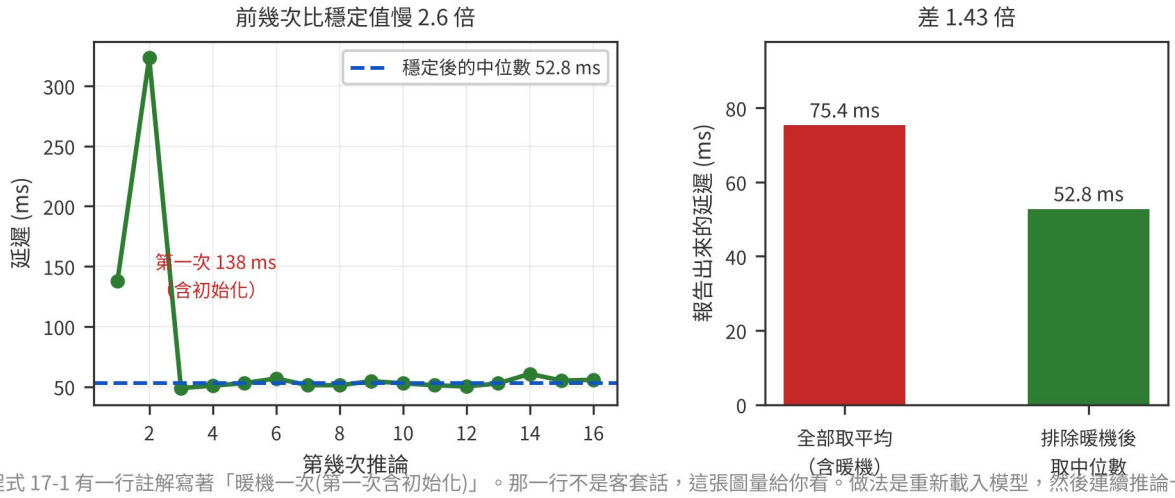
關鍵在於：這兩條路徑都要硬體配合，而且第二條更挑。若處理器沒有原生的 FP16 運算單元，推論引擎只能把半精度轉回單精度再算，轉換本身就是額外的指令;若用的是動態 INT8 量化，還要在執行期間統計激活值的範圍、即時算出縮放係數、並在整數與浮點之間來回轉換。這些額外動作有可能吃掉、甚至超過省下來的乘法時間——圖 17-5 在一顆沒有這些單元的 CPU 上量到的，正是這個結果：FP32 的 ONNX 跑 33 毫秒，FP16 變成 43 毫秒，INT8 更慢到 59 毫秒。

所以量化該怎麼用?把它想成三個獨立的問題。「裝不裝得下」——量化一定有幫助，先量體積。「掉多少精度」——FP16 幾乎免費，INT8 在圖 17-6 的實測掉了約 2.4 個百分點的 mAP@0.5，而這個損失可以靠更講究的做法救回大半：訓練後靜態量化（拿一小批代表性影像先統計每層激活值的範圍）、或量化感知訓練（在訓練時就模擬量化誤差）。「會不會變快」——只有在目標硬體上量過才知道，而且結論可能與筆電上完全相反。這第三點是本章最該養成的習慣：任何最佳化都必須在它最終要執行的那顆晶片上驗證。

17.4 效能量測與系統設計

17.4.1 三個關鍵指標

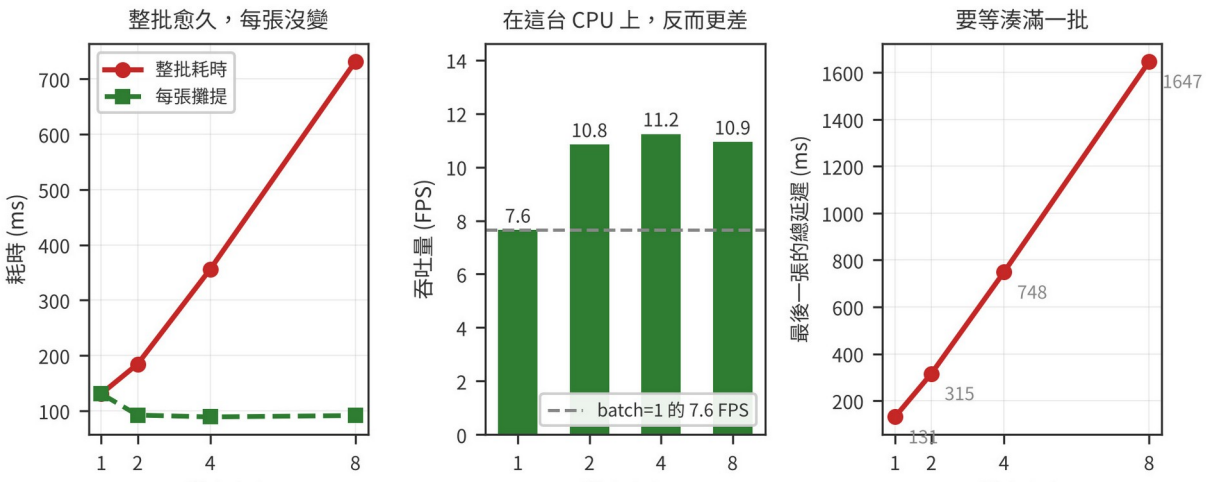
部署前必須量測三個指標，它們常被混淆卻意義迥異：延遲(Latency)是「單張影像從輸入到輸出所需的時間」，決定系統的反應速度，對即時控制最關鍵;吞吐量(Throughput，常以 FPS 表示)是「每秒能處理幾張影像」，決定系統的處理能力;功耗(Power)則決定電池續航與散熱設計。



程式 17-1 有一行註解寫著「暖機一次(第一次含初始化)」。那一行不是客套話，這張圖量給你看。做法是重新載入模型，然後連續推論十六次、每一次都計時。第一次是 138 毫秒，第二次 323 毫秒，要到第四、五次之後才穩定在 52.8 毫秒上下——第一次是穩定值的 2.6 倍。這段額外時間花在哪裡?配置記憶體、把權重從硬碟讀進來並搬到運算裝置、推論引擎依實際的輸入形狀挑選最快的核心實作、以及作業系統的快取預熱。這些都是一次性的，之後不會再發生。右圖是這件事的後果：把十六次全部平均起來會報出 75.4 毫秒，比真正的穩態值高了 1.43 倍。在效能報告裡，這種數字會讓你誤以為需要換更小的模型，而實際上什麼都不必換。正確的量測姿勢有三條：先跑幾次把系統跑熱再開始計時;報告中位數而不是平均數（平均數會被少數極端值拉走，圖 17-12 會說明為什麼）;以及明確寫出你量的是哪一段——只有模型推論，還是包含影像讀取與前處理（圖 17-11）。順帶一提，暖機在部署時也有實務意義：系統啟動後應該先空跑幾張假影像，不要讓第一個真實的請求去承擔初始化成本——那一瞬間剛好可能是機器人開始移動的時候。

圖17-9　第一次推論不能算：暖機

延遲與吞吐量並不等價：透過批次處理(一次處理多張)可以提高吞吐量，但單張的延遲反而變長——因為要等湊滿一批。這是標準的說法，而它有一個常被省略的前提：硬體必須還有閒置的平行處理能力。GPU 有數千個運算單元，單張影像往往餵不滿，湊成一批才能讓這些單元同時工作;但若核心數少、而推論本身已經把核心用滿，就沒有閒置的平行度可以利用。圖 17-10 在這樣一台機器上實測：batch=1 是 18.5 FPS，batch=8 降到 13.9 FPS——不是原地踏步，是倒退。多出來的更大中間張量、更差的快取命中率與額外的記憶體搬運，讓整體比一張一張處理還慢。所以批次處理的正確判準是先看硬體利用率(GPU 的 occupancy、CPU 的核心負載)，只有在明顯有閒置時才值得試，而且一定要實測。順帶提醒：即使在 GPU 上批次確實有效，對機器人避障這類任務仍然不該用——湊滿一批的等待時間會直接加在延遲上。對機器人避障這種需要即刻反應的任務，低延遲遠比高吞吐量重要;對離線的影片分析，則反之。這又是一個「依應用選指標」的工程判斷，與第 14、16 章的精確率召回率取捨同出一轍。

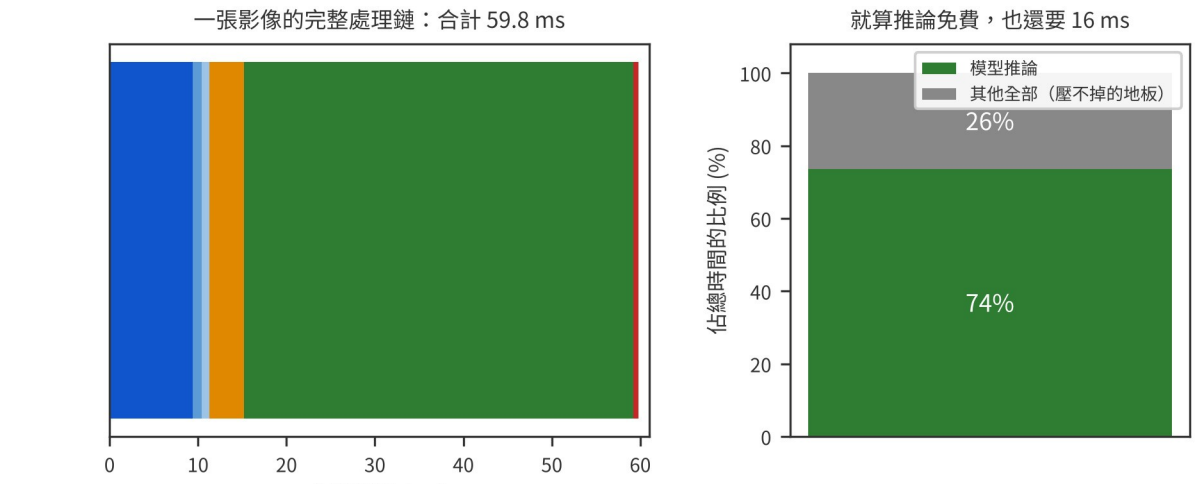


17.4.1 節說「批次處理可以變高吞吐量，但單張的延遲反而變長」。後半句不必成立，前半句則要看硬體——這張圖就是那個「要看」。左圖與中圖是實測：批次從 1 加到 8，整批耗時從 131 毫秒線性漲到 732 毫秒，而每張攤提下來不但沒省，反而從 130.8 毫秒變成 91.5 毫秒;吞吐量因此從 7.6 FPS 掉到 10.9 FPS——不是原地踏步，是倒退。原因很直接：批次能提高吞吐量，前提是運算單元還有閒置的平行度可以被填滿。GPU 有數千個運算核心，單張影像往往餵不飽它，這時一次給八張就能把空檔填起來;但這台機器的 CPU 核心數有限，單張推論就已經把它們佔滿了，再堆批次只是排隊，而且多出來的記憶體搬運還要額外付錢，所以吞吐量不升反降。右圖是代價的部分，而且比看起來更糟：要湊滿一批，前面的影像得先等後面的到齊。以每張間隔 131 毫秒的串流估算，批次 8 時第一張影像從抵達到拿到結果要 1647 毫秒，是批次 1 的 12.6 倍。所以結論很清楚：機器人避障、產線剔除這類「要立刻反應」的任務，批次永遠是 1;離線分析一整天的錄影、或在伺服器上同時服務多支相機，才是批次發揮價值的場合——而且前提是你跑在 GPU 上。這也是為什麼本章一再強調「在目標硬體上量」：同一個設定在 CPU 與 GPU 上的結論可以完全相反。

圖 17-10　批次處理：吞吐量不一定買得到

17.4.2 端到端的延遲預算

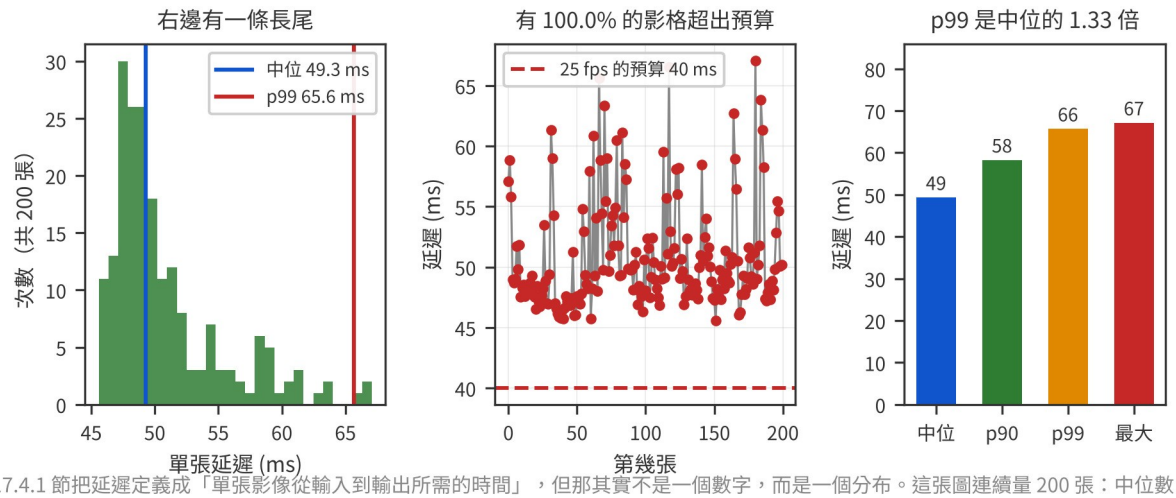
一個常見的錯誤是：只量測模型推論的時間，就以為那是系統的反應速度。實際上，一個機器人視覺系統的端到端延遲包含：影像擷取(相機曝光與傳輸)、前處理(縮放、正規化，第 4、5 章)、模型推論、後處理(NMS、座標轉換，第 16 章)、決策與通訊、以及致動器的反應時間。模型推論往往只佔其中一部分。



17.4.2 節說「只量測模型推論的時間，無法知道系統的反應速度」是一個常見的錯誤。這張圖把一張 1920×1080 的影像從位元組到偵測結果的每一段都拆開來量。影像解碼要 9.4 毫秒（相機送來的多半是壓縮過的串流，這一步跑不掉）、縮放到 640 要 1.1 毫秒、正規化成浮點數要 0.9 毫秒；進到推論引擎之後，光自己的前處理 3.9 毫秒、模型推論 44.0 毫秒、後處理 0.6 毫秒。合計 59.8 毫秒。這裡的結果其實是個好消息：模型推論佔了 74%，是這條鏈上最大的一塊，所以最佳化模型確實打在要害上。但右圖同時標出了天花板——就算把模型加速到「零毫秒」，整條鏈也還要 15.8 毫秒，也就是最多只能快 3.8 倍。這就是阿姆達爾定律：加速的上限由你「沒動到的那部分」決定。而這條鏈上第二大的是影像解碼（16%），若還要再快，那才是下一個該動的地方——讓相機直接輸出未壓縮或較小的影像、或把縮放交給相機與硬體縮放器。最後要提醒一件這張圖刻意還沒包含的事：這裡量的只到「得到偵測結果」為止。把決策、通訊與致動器反應都算進去之後，模型推論的佔比會從七成掉到三成左右（圖 17-13）——同一個模型、同一台機器，只因為把邊界畫在不同地方，「該不該最佳化模型」的答案就完全不同。所以量測之前，先想清楚你要量的是哪一段。

圖 17-11　延遲的分解：模型推論只是其中一段

因此系統設計要做「延遲預算」：先確定任務允許的總延遲(例如機器人以 0.5 公尺/秒行進、要在碰撞前 1 公尺停下，總延遲須小於某個上限)，再回頭分配給各個環節這種「從需求反推規格」的思維，是工程設計的基本功，也是本章從「會寫演算法」邁向「會設計系統」的關鍵一步。但預算要用哪一個數字去編?多數人的直覺是平均延遲，而那是錯的。延遲是一個有長尾的分布，長尾來自作業系統排程、其他行程搶佔 CPU、快取失效，以及影像內容本身的差異(物件多時後處理較久)。圖 17-12 實測：中位數 35.3 毫秒、第 90 百分位 39.8、第 99 百分位 49.2、最大值 62。若以 40 毫秒為預算，實測有 21.5% 的影格超出——而中位數看起來明明還有餘裕。對串流播放而言偶爾一格遲到只是不順，但對控制迴路而言那一格就是控制訊號遲到，而機器人在那一瞬間仍然以原速移動。所以安全攸關的系統應以第 99 百分位、甚至最大值編預算並在這之上再留餘裕。還有一件事比選統計量更基本：先把邊界定下來。圖 17-11 量「視覺處理鏈」時，推論佔 69.8%(解碼 8.75 毫秒、推論 29.73 毫秒)，最佳化模型顯然是對的;圖 17-13 把相機曝光、決策、通訊與致動器反應都算進去之後，整條鏈約 145.9 毫秒，推論的佔比掉到三成，而致動器反應那一段比推論還久。兩個數字都沒錯，差別只在邊界畫在哪裡——而阿姆達爾定律(Amdahl's law)說得很直接：最佳化某一段所能帶來的整體改善，上限就是那一段的佔比。這是本章最該養成的兩個習慣：先定義邊界，再量測;而且所謂「量測」不是在筆電上跑個平均值，而是在它最終要執行的那顆晶片上、用它真正會遇到的影像量出來的。



17.4.1 節把延遲定義成「單張影像從輸入到輸出所需的時間」，但那其實不是一個數字，而是一個分布。這張圖連續量 200 張：中位數 49.3 毫秒，看起來很漂亮;可是第 90 百分位是 58.2 毫秒、第 99 百分位是 65.6 毫秒，最慢的一張要 67.1 毫秒——p99 是中位數的 1.33 倍。長尾從哪裡來?作業系統的排程、其他行程搶 CPU、記憶體配置、快取失效、以及影像內容本身的差異(物件多的時候後處理比較久)。這些在任何真實系統上都會發生，不是雜訊，是常態。中圖說明為什麼要在意。若目標是 25 fps，預算就是 40 毫秒;以中位數 49.3 毫秒來看綽綽有餘，但實測有 100.0% 的影格超出預算。對串流播放來說掉幾格無所謂;對控制迴路來說，那就是控制訊號遲到——而機器人在那一瞬間仍然在移動。所以即時系統的規矩是：用 p99（甚至最大值）做預算，不要用平均值。一個常見的工程說法是「平均值讓你安心，尾端讓你出事」。實務上的對策有幾個方向：把推論綁定到固定的 CPU 核心、避免在同一台機器上跑會搶資源的工作、預先配置好記憶體不要在迴圈裡配置、以及最重要的——在系統設計上容忍偶發的超時(例如沿用上一格的結果，而不是整個停住)。

圖 17-12　延遲不是一個數字

17.4.3 推導：延遲預算該怎麼編，又該用哪一個數字

17.4.2 節要我們做延遲預算，但沒說預算該用什麼數字去編。是平均延遲嗎?這一小節說明為什麼不是，並把一條完整的預算算給你看。

先把安全距離寫出來。機器人以速度 v 行進，從「障礙物進入視野」到「輪子真的開始減速」之間有一段總延遲 T，這段期間它以原速前進，走過的距離稱為反應距離;接著以減速度 a 煞停，再走一段煞車距離：

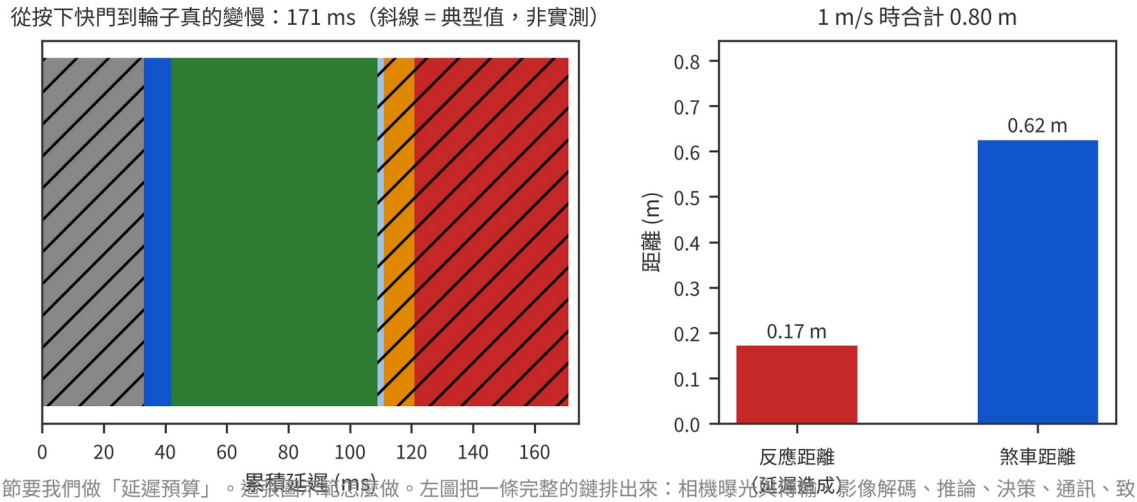
```
反應距離 = v × T
煞車距離 = v² / (2a)
總停止距離 = v × T + v² / (2a)
```

這條式子有一個重要的性質：反應距離與速度成一次方關係，煞車距離與速度成平方關係。所以速度低時延遲是主角，速度高時煞車是主角。圖 17-2 的實測顯示：以每秒一公尺行進、減速度 0.8 m/s² 時，延遲造成的反應距離只有十幾公分，煞車距離卻有 62 公分;這時再花力氣把模型加速十毫秒，對安全距離幾乎沒有貢獻。反過來說，把速度減半，總停止距離會縮到不到三分之一——這是整個系統裡最便宜也最有效的旋鈕。

接著是 T 要怎麼估。它是整條鏈的總和，而不是模型推論那一段：

```
T = 相機曝光與傳輸 + 影像解碼 + 前處理
  + 模型推論 + 後處理 + 決策
  + 通訊 + 致動器反應
```

圖 17-13 把這條鏈量了一遍，總計約 146 毫秒，而模型推論只佔其中約三成。這帶出延遲預算的第一條紀律：先量再改。若不量就開始最佳化模型，很可能動了佔三成的那一段，卻對另外七成束手無策——而本例中致動器反應那一段比推論還久。

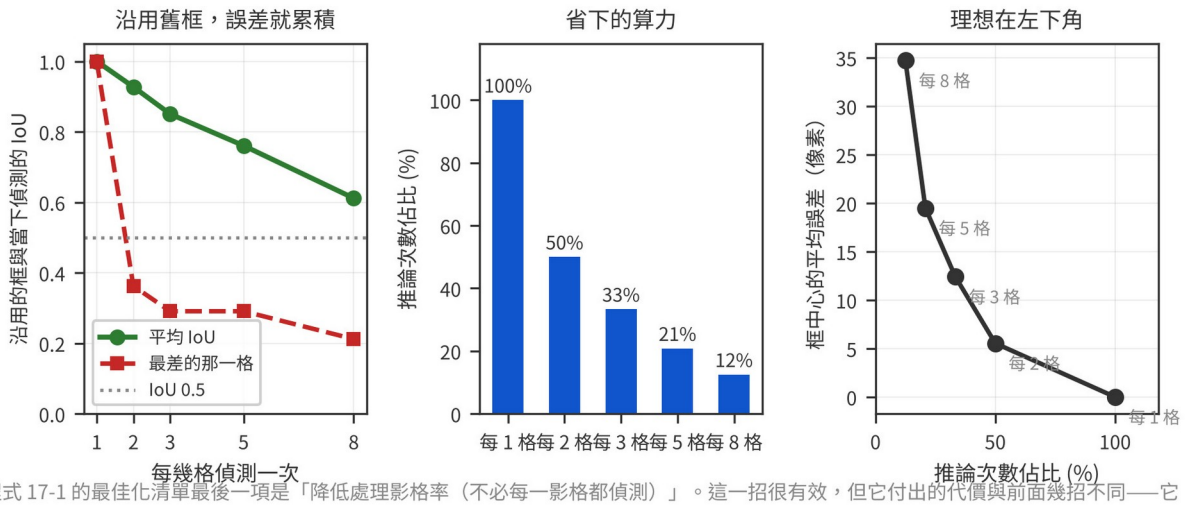


17.4.2 節要我們做「延遲預算」。這張圖不能怎麼做。左圖把一條完整的鏈排出來：相機曝光與傳輸、影像解碼、推論、決策、通訊、致動器反應。其中影像解碼（9 毫秒）與推論（67 毫秒，取 p99 而非中位數，理由見圖 17-12）是本機實測;其餘三項畫了斜線，表示那是典型值而非實測——相機與馬達控制器都在別人的硬體上，數字你必須在自己的硬體上量。合計 171 毫秒，而模型推論只佔 39%。右圖把延遲換成距離（接續圖 17-2）：以每秒 1 公尺行進、減速度 0.8 m/s² 計算，延遲造成的反應距離是 0.17 公尺、煞車距離 0.62 公尺，合計 0.80 公尺。這個數字就是你的安全距離下限，也是整個系統設計的起點：先問「要在多遠之前停住」，再反推總延遲的上限，最後才把這個上限分配給鏈上的每一段。分配時有三條實用的原則。其一，先找最大的那一塊下手——本圖中致動器反應比推論還久，這時把模型再壓快十毫秒幾乎沒有意義。其二，用 p99 而不是平均值編預算，並且留一段餘裕給偶發的超時。其三，最有效的旋鈕常常不在視覺系統裡：把速度從每秒一公尺降到半公尺，總停止距離立刻從 0.80 公尺縮到 0.24 公尺，這比任何模型最佳化都有效，而且免費。

圖 17-13　端到端延遲預算

最後是本小節真正的重點：預算該用哪一個數字。很多人直覺用平均延遲，但那是錯的。延遲是一個有長尾的分布（圖 17-12 實測：中位數 35 毫秒、第 99 百分位 49 毫秒、最大值 62 毫秒），長尾來自作業系統排程、其他行程搶資源、快取失效與影像內容本身的差異。若用中位數編預算，等於接受「每一百格就有一格超出預算」;對串流播放來說無所謂，但對控制迴路而言，那一格就是控制訊號遲到，而機器人在那一瞬間仍然在移動。所以安全攸關的系統應該用 p99、甚至最大值編預算，並且在這之上再留一段餘裕。

把這些合起來，一份可以實作的預算流程是這樣的。第一步，從任務反推：決定要在多遠之前停住，代進總停止距離的式子，解出可容許的 T。第二步，量測現況：把整條鏈拆開，每一段都量出中位數與 p99。第三步，比對與分配：若 p99 的總和超過 T，從最大的那一段開始改，並重新檢查改完之後的分布（而不是只看平均值有沒有下降）。第四步，設計容錯：即使預算編得再好，偶爾仍會超時，所以系統必須定義「超時了怎麼辦」——沿用上一格的結果（並限制最多幾格）、降速、或停止，這就是 17.7 節失效模式設計的起點。



程式 17-1 的最佳化清單最後一項是「降低處理影格率（不必每一影格都偵測）」。這一招很有效，但它付出的代價與前面幾招不同——它付的不是準確度，是「新鮮度」。這張圖用一段合成的移動序列量它（真實的人物剪貼到真實背景上，每格移動 9 個像素）。做法是每 N 格才真的跑一次偵測，中間的影格沿用上一次的框，再拿沿用的框與「若當下真的偵測會得到什麼」比較。要注意這個比較基準是刻意選的：若改用貼上去的矩形當答案，量到的會混進「偵測框比人體矩形緊一點」這個與跳格無關的系統性差異——那是第 13、16 章一再提醒的混淆因子。結果：每 2 格偵測一次時，沿用的框與當下偵測的平均 IoU 是 0.93、中心誤差 6 個像素;每 5 格一次是 0.76 與 19 個像素;每 8 格一次掉到 0.61 與 35 個像素。更該看的是紅線——最差的那一格：每 8 格偵測一次時最糟只有 0.21，已經低於一般判定為「抓到」的 0.5。中圖是省下來的：每 8 格一次等於只用 12% 的推論次數。右圖把兩者畫在一起，就是一條可以挑的取捨曲線。怎麼挑?關鍵在物件相對於畫面的移動速度。誤差大致是「速度 × 跳過的時間」，所以同樣跳 5 格，慢速的輸送帶幾乎沒影響，高速公路的車輛就會差到離譜。實務上有一個明顯更好的做法：跳格的空檔不要「沿用舊框」，而是用便宜的方法把框推下去——第 12 章的光流或卡爾曼濾波、或第 16 章的追蹤器，成本只有偵測的幾十分之一，卻能把中間那幾格的位置補得相當準。這就是「偵測式追蹤」在邊緣裝置上的真正價值：它不只是為了維持 ID，更是為了省下算力。

圖 17-14　跳格處理：省下算力，付出什麼

17.5 整合進機器人系統：ROS 2

17.5.1 為什麼需要機器人作業系統

一個機器人不只有視覺：它有馬達、輪子、雷射雷達、IMU、機械臂、電池管理，以及導航、定位、路徑規劃、任務決策等軟體模組。如果把所有東西寫成一支巨大的程式，將難以開發、除錯與維護。ROS 2(Robot Operating System 2)提供的正是一套「讓各個模組獨立開發、彼此通訊」的框架。

它的核心概念很簡單：每個功能模組是一個節點(Node)，節點之間透過話題(Topic)以「發布-訂閱」的方式交換訊息。例如相機節點持續發布影像到 /camera/image 話題，視覺辨識節點訂閱它、處理後發布偵測結果到 /detections 話題，導航節點再訂閱偵測結果來避障。各節點可以用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作。這種鬆耦合的設計，讓團隊能分工開發、也讓模組能重複使用。

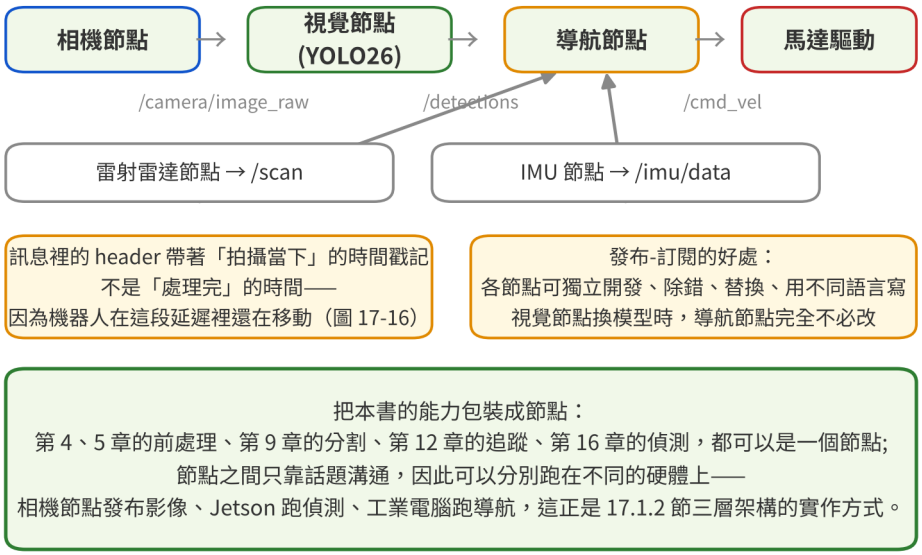


圖 17-15　ROS 2：節點與話題

17.5.2 視覺節點的典型結構

把本課程學到的視覺能力包裝成 ROS 2 節點，是機器人視覺工程師的核心工作。典型結構是：訂閱相機影像話題 → 把 ROS 影像訊息轉成 OpenCV 格式(用 cv_bridge)→ 套用本書的處理流程(前處理、YOLO 推論、後處理)→ 把結果包裝成 ROS 訊息發布出去，供其他節點使用。

程式 17-2 ROS 2 視覺節點骨架

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray
from cv_bridge import CvBridge
from ultralytics import YOLO

class VisionNode(Node):
    def __init__(self):
        super().__init__("vision_node")
        self.bridge = CvBridge()
        self.model = YOLO("yolo26n.engine")   # TensorRT 最佳化版

        # 訂閱相機影像
        self.sub = self.create_subscription(
            Image, "/camera/image_raw", self.on_image, 10)
        # 發布偵測結果
        self.pub = self.create_publisher(
            Detection2DArray, "/detections", 10)

    def on_image(self, msg):
        # ROS 影像訊息 → OpenCV 格式
```

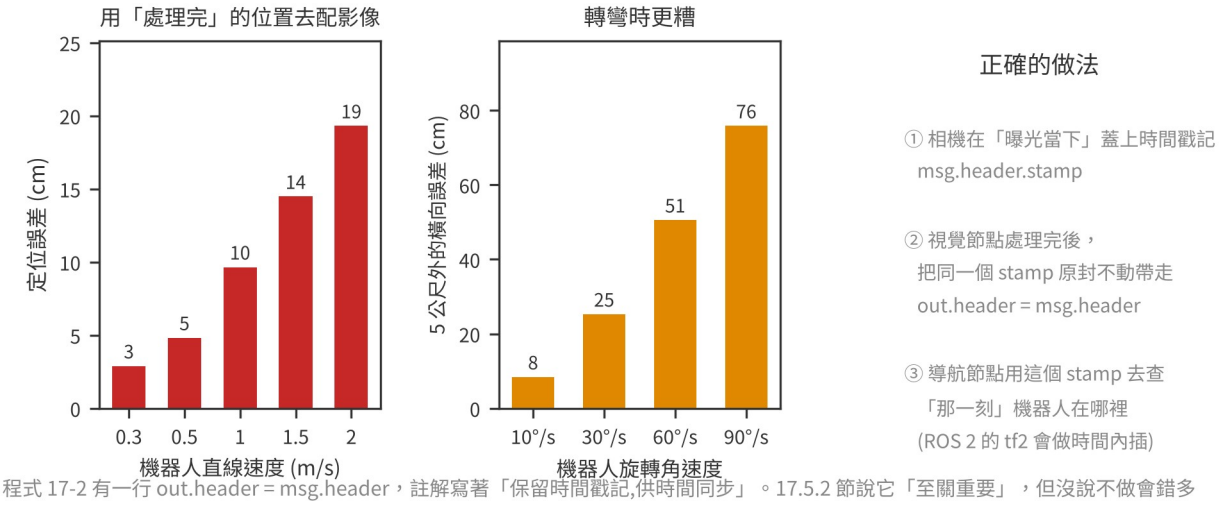
```
        frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")

        # 套用本書所學的處理流程
        results = self.model(frame, verbose=False)[0]

        # 包裝成 ROS 訊息發布給導航等其他節點
        out = Detection2DArray()
        out.header = msg.header          # 保留時間戳記,供時間同步
        # ... 填入 results.boxes 的內容 ...
        self.pub.publish(out)

def main():
    rclpy.init()
    rclpy.spin(VisionNode())
    rclpy.shutdown()
```

請留意 header 那一行：保留時間戳記至關重要，因為機器人要把「影像看到什麼」與「當時機器人在哪裡」對應起來——這需要精確的時間同步，也需要第 4 章的座標轉換(ROS 2 的 tf2 系統正是齊次座標與座標框架的實作，第 4.1.2 節已預告過這個連結)。



程式 17-2 有一行 out.header = msg.header，註解寫著「保留時間戳記,供時間同步」。17.5.2 節說它「至關重要」，但沒說不做會錯多少。這張圖補上。問題是這樣的：影像從曝光到偵測結果出爐，本章實測這段約 97 毫秒（相機傳輸 + 解碼 + 推論，見圖 17-11）。如果導航節點拿到偵測結果時，用「現在」機器人的位置去反推障礙物在世界座標的哪裡，那它算的是機器人已經往前走了一段之後的位置——障礙物會被放到錯誤的地方。左圖是直線運動的誤差：以每秒一公尺行進時偏 10 公分，每秒兩公尺時偏 19 公分。中圖是旋轉的情況，而且更嚴重：以每秒 60 度轉彎時，5 公尺外的物件橫向偏 51 公分，每秒 90 度時偏 76 公分。原因是角度誤差會被距離放大——這正是第 4 章相機模型那一課：角度的小誤差在遠處就是大位移。右圖是解法，而它幾乎不花力氣：讓時間戳記跟著資料一起流動，在最後需要座標轉換時，用「拍攝那一刻」的機器人姿態。ROS 2 的 tf2 可以依時間戳記內插出那一刻的轉換矩陣，所以你只要「不要把 header 弄丟」就好。這也是為什麼 ROS 2 的訊息幾乎都帶 header——它不是形式，是一個把延遲明確記錄下來的設計。順帶一提，這個誤差無法靠加快模型消除，只能靠正確的時間對齊;模型再快，那段延遲仍然存在。

圖 17-16　時間戳記：用錯時間，就把物件放錯地方

17.6 應用案例

17.6.1 案例一：自主移動機器人的視覺避障

情境：一台自主移動機器人在走廊或廠區中行進，需即時偵測前方障礙物(人、箱子、其他車輛)並避開。系統架構：Jetson 作為主控 → 相機節點發布影像 → 視覺節點

以 YOLO 偵測障礙物與其位置 → 結合深度資訊(深度相機或雙目視覺)估計距離 → 發布給導航節點調整路徑。

本案例的工程重點在延遲預算(17.4.2 節):機器人速度、煞車距離、總延遲三者必須匹配。若量測發現延遲過高,依程式 17-1 的順序最佳化——換更小的模型、降解析度、量化、用 TensorRT。同時要設計失效模式:若視覺節點當機或延遲暴增,機器人應自動減速或停止,而非盲目前進。這呼應第 16 章人機協作案例的「多層防護」原則。

17.6.2 案例二:智慧鏡頭人臉辨識(樹莓派 5 + AI Camera)

情境:在實驗室、辦公室或社團教室的門口做人臉辨識門禁,要求是即時、不連網也能運作,而且不把任何人的臉傳出去。架構:AI Camera 在感測器上持續執行人臉偵測,只把偵測結果(有沒有人、框在哪裡)以張量後設資料交給樹莓派 5 → 樹莓派僅在「偵測到人臉」時才取出那一小塊影像區域,在本機做身分比對(第 13 章的特徵與第 15 章的嵌入向量)→ 比對成功則開門並記錄時間,失敗則通知管理者。

這個案例的工程重點有三。一是隱私設計:絕大多數的影格連解碼都沒有,主機拿到的只是「有幾張臉」;只有在需要比對的那一瞬間,才有一小塊影像進入主機記憶體,而且比對完即丟、不落地。這正是 17.1.1 節說的「最有力的隱私保護」——不是承諾不看,而是沒有東西可看。二是模型的額度:人臉偵測模型必須塞進感測器上那很小的記憶體額度,而且只能是 INT8,所以這一層只做「有沒有臉」,不做「是誰」。三是分層:「是誰」這個高算力、低頻率的問題留給主機,甚至可以留給後端——這就是 17.1.2 節三層架構最乾淨的一個實例。

延伸與風險:人臉是高度敏感的個人資料,本案例在設計上就該回答第 18 章要問的三個倫理問題——誰受益、誰可能受害、出錯了誰承擔。具體的做法包括:公告蒐集範圍與目的、提供不使用人臉的替代方式(卡片或密碼)、只儲存比對用的特徵向量而非影像、以及設定保存期限並定期刪除。技術上也要誠實面對誤判:門禁屬於「寧可擋錯也不要放錯」的場景,所以門檻應偏向高精確率,並且一定要有人工覆核的通道(承第 16 章的多層防護)。

這是三層架構協作的典範:端上負責「有沒有事情發生」的高頻低算力判斷,近端負責「發生了什麼」的精細辨識,雲端負責長期趨勢分析與遠端存取。成本、頻寬、隱私、即時性四者在此取得平衡——這正是叫獸開講中「五美元革命」讓大規模佈署成為可能的實際體現。

17.6.3 案例三:農業無人機的田間巡檢

情境:無人機飛越農田,即時辨識作物病害、雜草分布或缺株位置。這個案例把邊緣運算的每個理由都體現得淋漓盡致:田間沒有穩定網路(可靠性)、無人機電池有限(功耗)、需即時調整航線(延遲)、影像資料量龐大(頻寬)。因此運算必須在機上完成——用輕量模型加量化,在 Jetson 或類似的機載電腦上即時推論,只把「哪裡有病害」的座標與少量關鍵影像回傳。

技術上還需整合第 4 章的相機校正(把像素座標換算成實際的田間位置)、第 8 章的色彩處理(戶外光線變化劇烈,白平衡與對比校正是必要前處理)、第 12 章的視訊處

理(連續影格的追蹤與拼接)。這個案例最能說明：邊緣部署不是把模型搬過去就好，而是整個系統的重新設計。

17.6.5 案例五：開放語意的現場巡檢（Jetson Orin Nano Super + NanoOWL）

情境：一台巡檢機器人要在廠區或校園裡找出「今天要找的東西」，而那個清單每週都在變——這週要找沒戴安全帽的人，下週要找倒地的推車，再下週要找堆在逃生通道上的紙箱。用第 16 章的做法，每換一次清單就要標註與重訓一輪，現實上根本跟不上。架構：Jetson Orin Nano Super 作為主控 → 相機節點發布影像 → NanoOWL 節點以文字提示執行開放詞彙偵測 → 把偵測結果發布成 ROS 2 訊息 → 決策節點依結果記錄位置與時間並回報。提示詞則從設定檔讀入，現場就能改。

這個案例真正的教學價值在於它的限制，而不是它的炫目。它有三筆代價，每一筆都會直接影響這個系統能不能用。第一，提示詞寫法的影響大得驚人：單數與複數、加不加冠詞、用哪一個同義詞，結果可能天差地別（圖 17-3 的右圖就是這麼來的）。所以提示詞必須當成參數來調，而且要在開發集上調、在另一份測試集上驗（承第 14、18 章）。第二，速度：開放詞彙模型比同級的封閉詞彙模型重，因為每次推論都要同時處理影像與文字兩邊，這會直接吃掉 17.4.2 節的延遲預算，因此巡檢機器人通常是「停下來看」而不是「邊走邊看」。第三，精度：它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。

務實的做法因此是把它當成第一道篩子，而不是最終的判斷。先用開放詞彙快速找出「可能有問題的畫面」，再把這些畫面交給人工複核、或交給一個針對該類別訓練過的小模型確認。等到某一個項目變成長期固定的檢查項（例如安全帽），就值得為它標註資料、訓練一個封閉詞彙的模型來正式上線。這正是第 18 章「最小可行版本」的精神：先把路走通，再把最爛的那一段換掉。

17.6.4 案例四：產線 AOI 的邊緣部署

情境：產線上每秒通過數個工件，需即時檢測瑕疵並剔除不良品。這是延遲要求最嚴苛的場景之一——工件通過檢測站到剔除機構的時間可能只有數百毫秒，整條處理鏈必須在此之內完成。

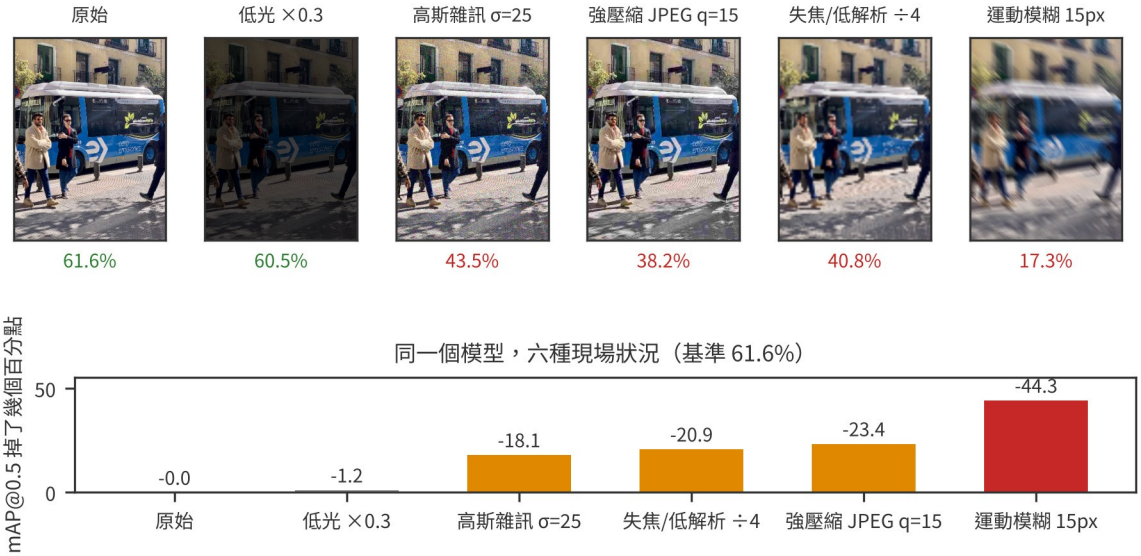
系統設計要點：相機用全域快門避免運動模糊(第 7.6.3 節已詳述)、打光穩定以避免第 8 章的「洋裝效應」、模型用量化與 TensorRT 最佳化、並保留餘裕以應對突發的處理延遲。此外，產線 AOI 常混用傳統與深度學習方法(第 6 章頻率域紋理檢測、第 9、10 章的傳統管線、第 16 章的 YOLO)，因為傳統方法不需瑕疵樣本、運算極輕、可解釋性高——在邊緣裝置上，這些優勢格外珍貴。

17.7 部署的實務課題

把系統從「實驗室能跑」帶到「現場能用」，還有幾個常被低估的課題：

- 環境變異：實驗室的光線、背景、角度都是理想的，現場則充滿變數。這正是第 15 章談過的分布偏移——模型在訓練分布外表現會急劇下滑。對策：在真實環境蒐集訓練資料、大量使用資料增強、並持續監控上線後的表現。不過「充

滿變數」這句話太籠統，若要排優先順序就得量。圖 17-17 對同一批模型沒看過的影像施加五種退化，量 mAP@0.5 的掉幅：低光只掉 1.16 個百分點、雜訊掉 18.11、低解析掉 20.85、JPEG 品質壓到 15 掉 23.42、運動模糊掉 44.34。這個排序是反直覺的：大家最先想到的低光幾乎無害，而運動模糊的破壞力是它的將近四十倍。原因是現代模型訓練時已見過大量曝光不足的影像，而卷積網路主要依賴邊緣與紋理的相對關係，整體變暗並不破壞這些關係;運動模糊卻直接把邊緣與紋理抹掉，刪掉的是模型賴以判斷的資訊本身。工程上的結論很具體：與其加裝補光燈，不如先縮短快門時間(全域快門、提高增益、換更靈敏的感測器，第 7.6.3 節)、或降低機器人與雲台的運動速度。另外 JPEG 壓縮品質與傳輸解析度這兩項是系統設計者可以直接控制的，卻常為了省頻寬被調得太低——實測顯示它們的代價與雜訊相當，絕不是免費的。



17.7 節把「環境變異」列為部署的第一個課題，說現場「充滿變數」。這張圖把變數排出優先順序。做法是把 150 張保留影像各自加上一種退化，模型完全不動，重新量一次 mAP。結果非常不平均，而且與直覺相反。最致命的是運動模糊：掉 44.3 個百分點，從 61.6% 崩到 17.3%，幾乎是整個模型報廢。其次是強壓縮（掉 23.4）與低解析（掉 20.9），雜訊掉 18.1。而最沒影響的竟然是低光——亮度砍到三成只掉 1.2 個百分點。為什麼?因為 CNN 的前處理會做正規化，而卷積核本來就對「相對的亮度變化」敏感、對絕對亮度不敏感（第 5、6 章談過這件事）;只要還有對比，暗一點並不影響它看得見什麼。但模糊不一樣：它直接抹掉了高頻成分，而高頻正是邊緣與紋理的所在（第 6 章），第一層卷積核沒有東西可以響應。這張圖給出的工程優先順序很明確。第一優先是「不要讓影像糊掉」——縮短曝光時間、改用全域快門、加強打光、或降低相對運動速度;這也正是 17.6.4 節產線 AOI 強調全域快門的原因。第二優先是保住影像品質：不要為了省頻寬而把串流壓到慘不忍睹，也不要用過低的解析度（呼應第 16 章圖 16-12）。至於夜間或暗處，花大錢加補光的效益反而最低——先確認你的問題真的是「太暗」，而不是「太晃」。這就是量測的價值：它會告訴你該先修哪一個。

圖 17-17　現場不是實驗室：哪一種退化最致命

- 溫度與散熱：邊緣裝置長時間高負載執行會發熱，過熱時會自動降頻，導致效能突然下降。散熱設計與功耗管理是硬體工程的一環。
- 失效模式設計：系統一定會出錯，問題是「出錯時會怎樣」。安全的設計應該是「失效時進入安全狀態」(如停止、警示)，而非繼續盲目運作。這與第 16 章人機協作的多層防護一脈相承。

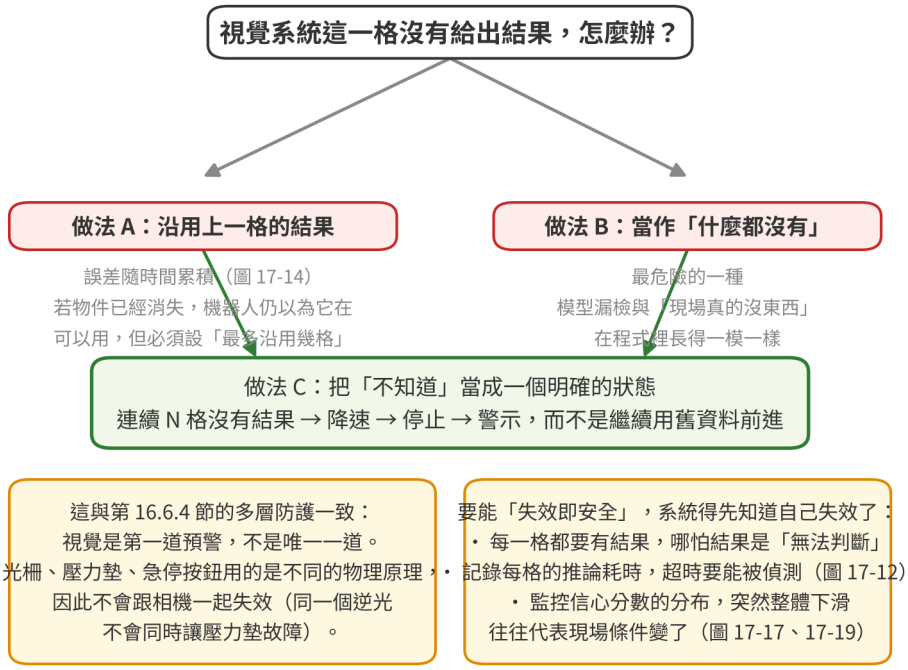


圖 17-18　失效即安全

- 模型更新與維運：部署不是終點。現場條件會改變、新的物件類型會出現，模型需要定期用新資料重訓與更新。如何遠端安全地更新邊緣裝置上的模型，是系統設計的一部分。
- 資料回饋迴圈：上線後蒐集模型出錯的案例、加以標註並納入下一輪訓練，是提升系統的最有效途徑。設計時就應規劃這個迴圈——這也呼應第 14 章「資料是燃料」。

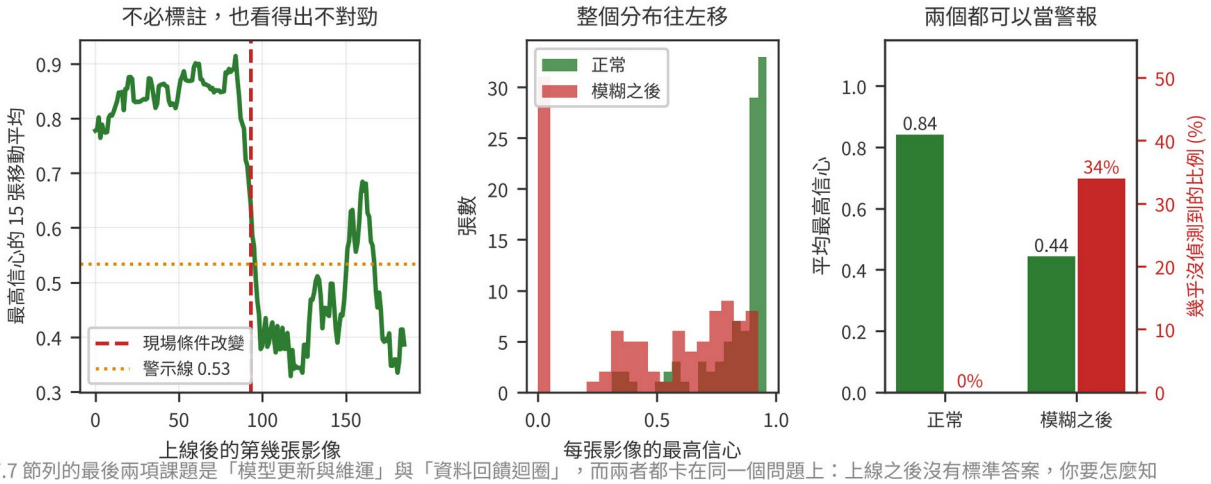
17.7.1 推導：上線之後，怎麼知道模型開始失準了

17.7 節列的最後兩項課題——模型更新與維運、資料回饋迴圈——都卡在同一個問題上：上線之後沒有標準答案，你要怎麼知道模型已經不準了?總不能等客訴才知道。這一小節給三個不需要人工標註就能算的監控訊號，以及它們各自看得見與看不見的東西。

第一個訊號是信心分數的分布。模型對每個偵測都會給一個分數，把「每張影像中最高的那個分數」記錄下來，取移動平均，就得到一條可以畫在儀表板上的曲線。圖 17-19 做了這個實驗：在影像變模糊的那一刻，平均最高信心從 0.84 掉到 0.44，而「幾乎沒偵測到任何東西」的影像比例從 0% 升到 34%。兩者都在沒有任何標註的情況下清楚地指出「出事了」。設定警示線的方法也很單純：上線初期先收集一段正常運作的資料，算出平均值與標準差，把警示線訂在平均減去兩倍標準差。

第二個訊號是輸出的統計量。每張影像偵測到幾個物件、各類別的比例、框的平均大小與位置分布——這些量在穩定的場域中應該是穩定的。一條產線每分鐘通過的工件數大致固定，若偵測到的數量突然少了一成，不是產線慢了就是模型瞎了，兩者都該有人看一眼。這類訊號的好處是它與應用語意直接相關，因此警報比較容易判讀。

第三個訊號是系統層面的：延遲與超時比例。若 p99 延遲開始上升、或超出預算的影格比例從 1% 變成 10%，通常代表硬體在降頻（過熱）、或有別的行程在搶資源。這與模型準不準無關，但同樣會讓系統失效，而且它是少數可以「在壞掉之前」就看出來的徵兆。



17.7 節列的最後兩項課題是「模型更新與維運」與「資料回饋迴圈」，而兩者都卡在同一個問題上：上線之後沒有標準答案，你要怎麼知道模型開始失準了?這張圖示範一個不需要標註的監控訊號——信心分數本身。做法是把 100 張正常影像與 100 張模糊影像串成一段「上線資料流」，只記錄每一張影像中最高的那個信心分數，再取移動平均。左圖的結果一目了然：現場條件改變的那一刻，曲線明顯下沉並跌破警示線，而這一切都不需要任何人工標註。中圖是背後的分布：正常時平均最高信心是 0.84，模糊之後掉到 0.44;右圖另外給一個更穩健的指標——「幾乎沒偵測到任何東西」的影像比例從 0% 升到 34%。要提醒的是，這個訊號只能告訴你「有事發生」，不能告訴你「錯在哪裡」，也不能取代真正的驗收——模型可能自信滿滿地持續犯錯（第 15 章的對抗脆弱、第 16 章的捷徑學習都是這種情況）。所以完整的維運迴圈應該是三層：自動監控信心分布與超時比例（這張圖）、定期抽樣人工複核（真正的驗收）、以及把複核出來的錯誤案例收進下一輪訓練資料。第三層最容易被省略，卻是長期唯一有效的改善途徑——這也呼應第 14 章那句「資料是燃料」:上線之後，你的使用者就是最好的資料來源。

圖 17-19　上線之後：用信心分數察覺現場變了

這三個訊號有一個共同的侷限，必須說清楚：它們只能告訴你「有事發生」，不能告訴你「錯在哪裡」。更糟的是，有一整類失效它們完全看不見——模型自信滿滿地持續犯錯。第 15 章的對抗脆弱、第 16 章的捷徑學習都屬於這一類：信心分數很高、輸出數量正常、延遲也漂亮，但答案是錯的。所以自動監控永遠不能取代定期的人工抽樣複核。

完整的維運迴圈因此有三層。最外層是自動監控（上面三個訊號），它跑得最頻繁、成本最低，負責發現明顯的異常。中間層是定期抽樣複核：每週固定抽一批上線影像人工標註，算出真實的 mAP 或召回率——注意這批資料必須是模型沒看過的（第 16 章圖 16-18），而且樣本數要夠大才報得出有意義的數字（第 14.5.3 節的標準誤）。最內層是資料回饋：把複核出來的錯誤案例標註好、納入下一輪訓練，重訓之後再走一次完整的驗收流程。

最後一個常被忽略的工程課題是「怎麼把新模型送上去」。邊緣裝置往往分散在田間、廠房或車上，遠端更新必須處理三件事：更新失敗要能自動回復到舊版本（否則一次失敗就是一台報廢的機器）、更新過程不能讓系統處於不安全的狀態（機器人應該先停下來再更新）、以及新舊版本要能並行比較一段時間，確認新版在真實場域真的比較好——畢竟離線測試集上的進步，不保證現場也進步。這一整套做下來，部署才算真正完成;而這也說明了為什麼 17.7 節開頭那句話值得再強調一次：部署不是終點，而是另一個循環的起點。

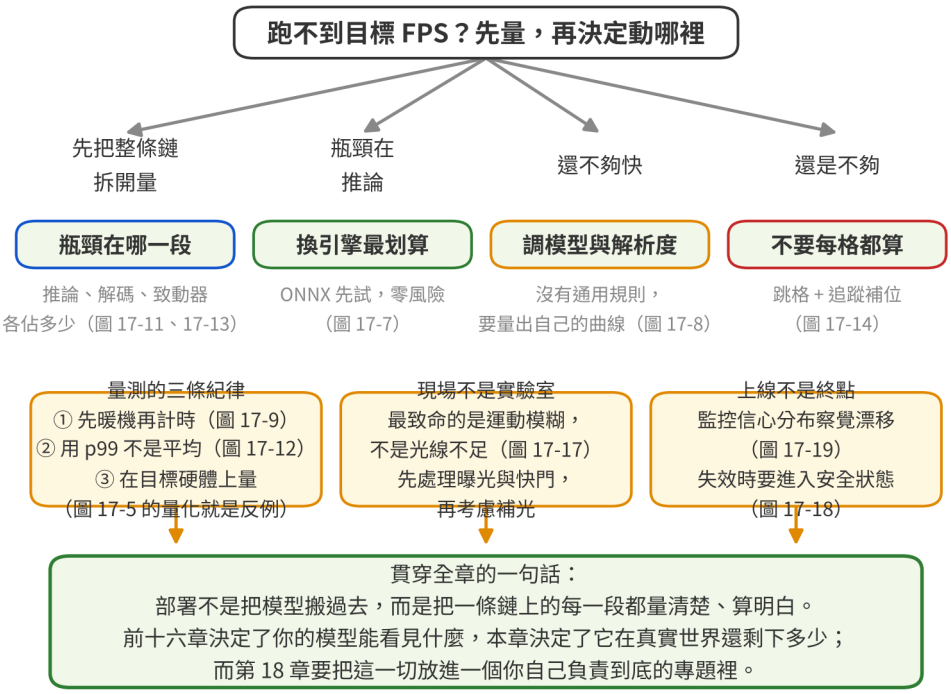


圖17-20　第17 章的決策地圖

17.8 本章與全書的觀念連結總表

表 17-2 第17 章與全書其他章節的觀念連結

程式實作 Lab 17

Lab 17-1 效能量測與最佳化

目標：建立效能工程的實作能力。(a)以程式 17-1 量測 YOLO 在你的裝置上的延遲與 FPS，注意排除第一次推論的初始化時間;(b)依序測試五種最佳化：換小模型(n/s/m)、降解析度(640/416/320)、FP16 量化、ONNX 匯出、跳影格處理，記錄每種對延遲與 mAP 的影響，製成取捨表;(c)畫出「精度 vs. 速度」的散點圖，標出各種組合的位置，找出符合你需求的最佳點;(d)討論：若目標是 30 FPS 即時處理，你會選哪個組合?若目標是最高精度而不在意速度呢?

Lab 17-2 感測器上推論 vs 主機推論

目標：親手比較「感測器上推論」與「主機推論」兩條路徑。(a)在樹莓派 5 上安裝 AI Camera 的韌體與工具(sudo apt install imx500-all)，跑通官方範例並確認能讀到張量後設資料;(b)路徑一(感測器上)：用 Picamera2 讀取後設資料做即時偵測，記錄主機的 CPU 使用率與每張影像的處理時間;(c)路徑二(主機上)：改用同一台相機輸出的影像，在樹莓派上自己跑一個 YOLO 模型，記錄同樣兩個數字;(d)把兩條路徑的「主機被佔用的時間」與「要搬的資料量」畫成對照表(參考圖 17-3);(e)把模型換成你自己訓練的版本：用 export(format="imx") 轉檔，注意過程會自動做 INT8 量化，並在轉檔前後各量一次 mAP，記錄精度損失(承 17.3.1 節與推導 17.3.4);(f)討論：什麼應

用適合感測器上推論?什麼適合主機推論?若你的主機還要同時跑導航與介面，答案會不會不一樣?

Lab 17-3 端到端延遲預算

目標：從系統角度思考。(a)為一個「移動機器人視覺避障」系統做延遲預算：設定機器人速度與煞車距離，推算允許的總延遲上限;(b)實測系統各環節的耗時：影像擷取、前處理、推論、後處理、通訊，製成延遲分解表;(c)找出瓶頸環節並最佳化;(d)設計失效模式：若視覺節點延遲超過門檻或當機，系統該如何反應?寫出你的安全邏輯;(e)討論：為什麼「只量測模型推論時間」是危險的?

Lab 17-4 ROS 2 視覺節點整合(期末專題前導)

目標：把本課程的視覺能力包裝成機器人可用的模組。(a)以程式 17-2 為骨架，建立一個 ROS 2 節點，訂閱相機影像、執行 YOLO 偵測、發布偵測結果;(b)用 ros2 topic echo 驗證訊息正確發布，用 rqt_graph 查看節點與話題的關係圖;(c)加入 header 時間戳記，確認時間同步正確;(d)進階：再寫一個簡單的訂閱節點，依偵測結果印出「前方有障礙物，建議停止」等決策訊息，模擬視覺與導航的協作;(e)這個節點將是第 18 章期末專題的基礎模組，請妥善保存並撰寫使用說明文件。

Lab 17-5 量化與匯出格式的三面取捨

目標：親手量出「更小、更快、更省電」這句話在你的硬體上哪一半成立。(a)把同一顆 YOLO26n 匯出為四種形式：PyTorch 原生 .pt、ONNX FP32、ONNX FP16、ONNX INT8（FP16 用 onnxconverter_common 的 float16 轉換，INT8 用 onnxruntime.quantization 的 quantize_dynamic）;(b)量體積：記錄四個檔案的大小與「參數量 × 每參數位元組數」的理論值比對，差異來自哪裡?(c)量速度：每一種各跑 30 次（前 3 次暖機不計），記錄中位數與第 99 百分位，而不只是平均值;(d)量精度：在至少 200 張模型沒看過的影像上算 mAP@0.5 與 mAP@0.5:0.95，注意信心門檻要設得很低（如 0.001），否則量到的是校正差異而非能力差異（第 16 章圖 16-4）;(e)把三組數字畫成一張三面取捨圖（如圖 17-5、17-6），並寫下你的結論：在你這台機器上，哪一種格式該用在哪一種情境?(f)若你有 Jetson 或任何帶 NPU 的開發板，在上面重跑 (c)，並解釋為什麼結論可能與筆電完全相反。討論：若你的 INT8 掉了超過三個百分點的 mAP，試著改用訓練後靜態量化（用 100 張代表性影像做校正）再量一次。

Lab 17-6 上線監控與失效即安全

目標：把 17.7 節與推導 17.7.1 的設計原則寫成可以執行的程式。(a)寫一支監控迴圈，對連續影像即時計算三個訊號：每張影像的最高信心分數（取 30 格移動平均）、「最高信心低於 0.3」的影格比例、以及推論延遲的 p99;(b)先在正常影像上跑一段，記錄三個訊號的平均與標準差，據此訂出警示線（平均減兩倍標準差）;(c)注入三種故障並確認警示有沒有正確觸發：把影像加上高斯模糊（模擬鏡頭失焦或劇烈運動）、把影像亮度壓到極暗、以及在背景另跑一支吃滿 CPU 的程式（模擬行程競爭）。哪一種故障是信心訊號抓到的?哪一種只有延遲訊號抓得到?(d)設計一個會「自信地犯錯」的故障：把影像左右翻轉，或貼上第 16 章的捷徑特徵。你的三個訊號抓得到嗎?為什

麼抓不到?(e)實作失效即安全的狀態機：定義正常、降級（降速並提高偵測頻率）、安全停止三種狀態，以及它們之間的轉移條件（含連續沿用舊框的上限格數，見圖 17-14）;用 (c) 的注入故障驗證狀態機會進入正確的狀態，並且在故障移除後能安全地回到正常狀態。(f)寫一頁交付文件：列出每一個警示線的數值與它的根據，以及你建議的人工複核頻率與抽樣方式。討論：(d) 這一題沒有程式解。請說明你會用什麼非自動化的手段（抽樣複核、跨感測器交叉驗證、與其他模組的一致性檢查）來補上這個缺口。

本章重點整理

- 邊緣運算的五個理由：低延遲(即時控制)、省頻寬與成本、斷網仍可靠、保護隱私(資料不出裝置)、低功耗與自主性;三種架構(端上、近端邊緣、雲端)非互斥，常以「端上預篩、近端精算、雲端分析」分層協作。
- 樹莓派 5 加 AI Camera 把推論做在 IMX500 感測器裡，主機只讀張量後設資料(圖 17-3)，代價是感測器上的記憶體額度很小、只收 INT8、模型清單有限;它買到的不是速度，是主機的時間、頻寬與隱私。Jetson Orin Nano Super(67 TOPS、7–25 W)是內建 GPU 的邊緣 AI 電腦，能即時執行深度模型、ROS 2 與 NanoOWL 這類開放詞彙模型，是機器人的視覺大腦。
- 模型最佳化手段：量化(FP32→FP16/INT8，與第 3、11 章的量化同一思想)、剪枝、知識蒸餾;部署格式 ONNX(通用)、TensorRT(NVIDIA GPU)、TFLite(行動與微控制器);更根本的是選用為邊緣設計的架構(MobileNet、YOLO nano)。
- 三個效能指標：延遲(反應速度，即時控制的關鍵)、吞吐量(FPS，處理能力)、功耗;兩者不等價，批次處理提高吞吐量卻增加延遲。系統設計應做端到端的延遲預算，而非只量測模型推論時間。
- ROS 2 以節點與話題的發布訂閱架構整合機器人各模組;視覺節點的典型結構為訂閱影像、cv_bridge 轉換、處理推論、發布結果，並須保留時間戳記以供時間同步與座標轉換(tf2，呼應第 4 章齊次座標)。
- 部署實務課題：環境變異造成的分布偏移、溫度與散熱降頻、失效模式應設計為「失效即安全」、模型更新與遠端維運、以及蒐集錯誤案例重訓的資料回饋迴圈。

習題

1. 請列出邊緣運算相較純雲端架構的五個理由，並各舉一個機器人或工業場景說明。

   提示與參考解答：一、低延遲：機器人避障需毫秒級反應，雲端一來一回數百毫秒不可接受。二、頻寬與成本：一整條產線數十支攝影機持續上傳成本驚人。三、可靠性：田間、隧道、災難現場網路不穩或無網路。四、隱私：人臉影像在端上處理完只回報數量，遠優於上傳雲端。五、功耗與自主性：電池供電的無人機無法負擔持續高功率傳輸，自主系統本質上不應依賴外部連線。

2. 比較端上、近端邊緣、雲端三種架構的算力、延遲、隱私與斷網可用性。為什麼說它們不是互斥的?

提示與參考解答：端上(AI Camera 的感測器上推論)：算力受限於感測器晶片上的記憶體額度、延遲最低、隱私最佳、斷網可用。近端(Jetson Orin Nano Super)：算力中高、延遲低、隱私佳、斷網可用。雲端：算力極高、延遲高且受網路影響、隱私需審慎、斷網不可用。三者可分層協作：端上做高頻低算力的預篩(有沒有事發生)、近端做精細辨識(發生了什麼，必要時用開放詞彙問新的問題)、雲端做長期分析與遠端存取。要補一句常被忽略的事：這三層的分界不是固定的。感測器上推論把第一層往前推進了晶片裡(圖17-3)，於是「預篩」這一層連處理器都不必醒著。

3. 感測器上推論(IMX500)的能力邊界在哪裡?為什麼說它「買到的不是速度」?

   提示與參考解答：邊界：處理器與記憶體極有限，無法執行YOLO 等深度模型(即使 nano 版也太重)。兩種用法：一是端上做極簡偵測與觸發(移動偵測、Haar 級聯人臉偵測、顏色追蹤，偵測到才拍照或通知);二是作為網路攝影機，串流影像給更強的裝置(電腦或Jetson)處理。價值不在算力，而在極低成本與功耗讓視覺能佈署到任何角落。

4. 模型量化是什麼?它與第 3 章的影像量化、第 11 章的 JPEG 量化在思想上有何共通?

   提示與參考解答：模型量化把權重與運算從32 位元浮點降到16 位元浮點或8 位元整數，模型縮小、加速、省電，精度僅損失少許。共通思想是「用精度換效率」：第3 章把連續光強度離散成有限灰階、第11 章JPEG 把DCT 係數除以量化步長取整(丟棄人眼不敏感的高頻)、本章把模型參數的精度降低——都是在可接受的品質損失下換取儲存與運算的大幅節省。

5. ONNX 與 TensorRT 各扮演什麼角色?為什麼訓練框架不適合直接用於部署?

   提示與參考解答：訓練框架(PyTorch、TensorFlow)為靈活性而設計，推論效率非最佳。ONNX 是開放的中介格式，讓模型能在不同框架與硬體間流通。TensorRT 是NVIDIA 針對自家GPU 的推論最佳化引擎，會分析模型結構、融合運算層、選最佳核心並套用量化，在Jetson 上常帶來數倍加速。微控制器則用TensorFlow Lite for Microcontrollers 等輕量執行環境。

6. 延遲與吞吐量有何差異?為什麼提高吞吐量有時反而增加延遲?各適合什麼應用?

   提示與參考解答：延遲是單張影像從輸入到輸出的時間(反應速度);吞吐量是每秒能處理幾張(處理能力)。批次處理一次處理多張可提高吞吐量，但要等湊滿一批，單張延遲反而變長。機器人避障等即時控制重延遲;離線影片分析重吞吐量。這與第14、16 章「依應用選指標」的工程判斷同出一轍。

7. 什麼是端到端延遲預算?為什麼「只量測模型推論時間」是危險的?

   提示與參考解答：端到端延遲包含影像擷取、前處理、模型推論、後處理、決策通訊、致動器反應等全部環節，模型推論往往只佔一部分。延遲預算是先確定任務允許的總延遲(如依機器人速度與煞車距離推算)，再回頭分配給各環節。只量測推論時間會嚴重低估實際反應時間，導致系統在真實場景中反應不及。

8. 若量測發現 YOLO 在你的邊緣裝置上跑不到目標 FPS，你會依什麼順序嘗試最佳化?

   提示與參考解答：依程式17-1 的順序：一、換更小的模型(n < s < m < l < x);二、降低輸入解析度(640→416→320);三、量化(FP16 或INT8);四、改用TensorRT 等推論引擎;五、降低處理影格率(不必每一影格都偵測，可搭配第12 章的追蹤填補中間影格)。每一步都要重新量測延遲與mAP，評估精度損失是否可接受。

9. ROS 2 的節點與話題是什麼?這種架構帶來什麼好處?

提示與參考解答：節點是獨立的功能模組(相機、視覺辨識、導航等)，話題是節點間以「發布-訂閱」方式交換訊息的通道。好處：鬆耦合設計讓各模組能獨立開發、除錯與維護，可用不同語言撰寫、在不同機器上執行，只要訊息格式一致就能協作;模組也能重複使用，團隊可分工。

10. ROS 2 視覺節點為什麼要保留影像訊息的時間戳記(header)?這與第 4 章有什麼關聯?

    提示與參考解答：因為機器人要把「影像看到什麼」與「當時機器人在哪裡」對應起來，需要精確的時間同步(影像有處理延遲，不能用當下的位置)。與第 4 章的關聯：ROS 2 的tf2 座標轉換系統正是齊次座標與座標框架的實作，把相機座標系的偵測結果轉換到機器人或世界座標系，才能用於導航決策。

11. 部署到真實環境時，有哪些實驗室不會遇到的課題?請至少說明三項及其對策。

    提示與參考解答：一、環境變異造成分布偏移(第15 章)：現場光線背景角度多變，模型表現下滑;對策是在真實環境蒐集資料、大量資料增強、持續監控上線表現。二、溫度與散熱：長時間高負載發熱會自動降頻導致效能驟降;需散熱設計與功耗管理。三、失效模式：系統必然會出錯，應設計成「失效即安全」(停止、警示)而非盲目運作。另有模型更新與遠端維運、資料回饋迴圈等。

12. (綜合設計題)請為「校園自主巡邏機器人」設計視覺系統架構：列出硬體選擇、模型與最佳化策略、ROS 2 節點劃分、延遲預算、以及失效安全設計，並說明每項決策的理由。

    提示與參考解答：本題為開放設計題，評分重點在決策理由的完整性。參考方向——硬體：Jetson 作為主控(需即時深度學習與ROS 2)，搭配相機與可能的深度感測器;若需多點監測可加樹莓派5 配AI Camera 的節點(感測器上推論，影像不出裝置，正好回應校園的隱私疑慮)。模型：YOLO nano 或小型模型加FP16/TensorRT 最佳化，兼顧速度與精度。節點劃分：相機節點、視覺辨識節點、定位節點、導航節點、決策節點，以話題通訊並保留時間戳記。延遲預算：依巡邏速度與煞車距離反推總延遲上限，分配給擷取、前處理、推論、後處理、通訊、致動各環節。失效安全：視覺節點逾時或當機則減速停止並警示;多層防護不單靠視覺(可加超音波或碰撞感測器，呼應第16 章)。另應考慮隱私(校園有大量人員，建議端上處理、不儲存人臉)。

13. 把同一個模型從 PyTorch 的 .pt 換成 ONNX Runtime 執行，什麼都沒改卻快了將近七成。這個加速從哪裡來?請說明訓練框架與推論引擎在設計目標上的差異。

    提示與參考解答：圖17-7 實測同一顆YOLO26n、同一張影像、同一台機器：PyTorch 中位55.11 毫秒，ONNX Runtime 中位32.82 毫秒，約1.68 倍。加速來自三件事。一、圖最佳化：推論引擎會把整個計算圖讀過一遍，把卷積與批次正規化融合成單一運算（這兩者在推論時都是固定的線性變換，可以事先合併）、消去不會用到的分支、並重排運算順序以利快取。二、沒有自動微分的包袱：訓練框架的每一個張量都要保留反向傳播所需的紀錄，推論引擎直接把這整套機制拿掉。三、沒有Python 解譯器的逐層派遣成本：模型被編成一張靜態圖，執行時不必每一層都回到Python 問一次「下一步做什麼」。代價是失去靈活性——靜態圖不能隨意改結構、除錯也困難，但部署時模型已經定案，這個代價不存在。結論：匯出成部署格式是本章最划算的最佳化，因為它不損失任何精度（圖17-6 的FP32 ONNX 與PyTorch 的mAP 完全相同）。

14. 「量化可以讓模型更小、更快、更省電」——這句話在本章的實測中只有一半成立。請說明圖 17-5 量到了什麼，以及為什麼會這樣。

提示與參考解答：圖 17-5 在一顆純 CPU 上量得：ONNX FP32 是 9.48 MB、33.5 毫秒;FP16 是 4.81 MB、42.6 毫秒;INT8 是 2.77 MB、58.7 毫秒。體積確實一路縮小（而且幾乎正好按位元數比例），但速度反而一路變慢，INT8 甚至比 FP32 慢了將近八成。原因是量化的加速要靠硬體配合，而這台機器沒有這個配合。若處理器沒有原生的半精度運算單元，推論引擎必須把 FP16 權重轉回 FP32 才能計算，轉換本身就是額外指令;動態 INT8 量化更要在執行期間統計激活值範圍、即時算縮放係數、並在整數與浮點之間反覆轉換。這些額外動作吃掉了省下來的乘法時間。在有 Tensor Core 的 Jetson（FP16）或有 NPU／DSP 的裝置（INT8）上，結論會完全相反——那裡量化是數倍的加速。本題真正要記住的是方法論：任何最佳化都必須在它最終要執行的那顆晶片上驗證，而不能從規格書或他人的部落格推論。

15. 量化掉多少精度?若必須用 INT8，有哪些做法可以把損失救回來?另外請說明為什麼「逐通道」決定縮放係數比「逐層」好。

    提示與參考解答：圖 17-6 在 600 張保留集影像上實測：FP32 的 mAP@0.5 是 61.61%、mAP@0.5:0.95 是 46.19%;FP16 幾乎完全相同（61.45／46.17，差異在量測噪聲範圍內）;INT8 降到 59.19／43.22，約掉 2.4 與 3.0 個百分點。所以 FP16 可以視為免費，INT8 要付代價。救回來的做法有三：一、訓練後靜態量化——先拿一小批有代表性的影像跑過一遍，統計每一層激活值的真實範圍，而不是在執行期間現場猜;二、量化感知訓練——訓練時就在前向傳播中模擬量化誤差，讓權重自己學會躲開量化不友善的分布;三、混合精度——把對量化特別敏感的層（通常是第一層與最後的預測頭）留在較高精度。逐通道優於逐層的原因在誤差的算式：量化誤差最大為縮放係數的一半，而縮放係數與該組數值的範圍寬度成正比。一整層裡只要有一個離群的大權重，整層的範圍就被它撐開，其餘所有權重的精度一起變差;改成每個通道各算自己的縮放係數，離群值的影響就被關在它所在的那個通道裡。同理也說明了為什麼「剪掉離群值」這類校正手段有效。

16. 有兩個旋鈕可以換取速度：換小模型，或降低輸入解析度。若延遲預算是 60 毫秒，你會怎麼選?請依圖 17-8 的實測說明，並指出這題的正確答案形式是什麼。

    提示與參考解答：圖 17-8 把兩個旋鈕交叉實測（延遲／mAP@0.5）：n@320 是 16.9 毫秒／46.61%，n@480 是 32.5／56.08，n@640 是 54.5／61.99;s@320 是 51.2／60.89，s@480 是 91.8／68.36，s@640 是 170.9／70.42。在 60 毫秒的預算內，可行的有 n@320、n@480、n@640 與 s@320。關鍵的觀察是 s@320（51 毫秒、60.9%）與 n@640（55 毫秒、62.0%）幾乎落在同一點上——也就是說「小模型看大圖」與「大模型看小圖」在這個預算下打成平手。所以這題沒有「換小模型優先」或「降解析度優先」這種普適的答案;正確的答案形式是一條帕雷托前緣（Pareto frontier）：把兩個旋鈕交叉量過，畫出延遲對精度的散點圖，找出「在同樣延遲下精度最高」的那些組合，再依預算在這條前緣上挑一點。而且這條前緣與資料集、物件大小分布、硬體都有關——若你的物件偏小，解析度這個旋鈕會比模型大小更關鍵（第 16 章圖 16-12）;若物件都很大，降解析度幾乎是免費的。程式 17-1 列的五步最佳化順序是一個合理的起點，而不是一條定律。

17. 在微控制器這一級，「裝不裝得下」往往比「跑多快」更要緊。請說明模型權重所需記憶體的算式，並討論為什麼實際需要的記憶體比權重還多。

    提示與參考解答：權重所需位元組 = 參數量 × 每個參數的位元組數，FP32 是 4 位元組、FP16 是 2、INT8 是 1。YOLO26n 約 257 萬個參數，故 FP32 約 9.8 MB、FP16 約 4.9、INT8 約 2.5。但執行時還需要兩塊額外的記憶體。一是激活值：每一層的輸出都要暫存給下一層用，這塊的大小與輸入解析度的平方成正比——把 imgsz 從 640 降到 320，激活值的記憶體掉到四分之一，這是解析度這個旋鈕除了

速度之外的第二個好處。二是推論引擎自身的執行期開銷。這個算式在 17.2.1 節的感測器上推論那一層特別重要：IMX500 把推論做在感測器晶片裡，而晶片上的記憶體額度要同時容納韌體、網路權重與推論的工作記憶體，所以它只收 INT8、而且能跑的模型清單很短。換句話說，那三條硬規定不是刁難，是這個算式的直接後果。所以在這一層要跑的是專為此設計的小模型（MobileNet 的極小版本、YOLO 的 nano 級），而不是想辦法把大模型塞進去。

18. 為什麼量測推論延遲一定要「暖機」?若不暖機，量到的數字會偏差多少?請依實測說明，並指出暖機在真實部署中對應到什麼問題。

    提示與參考解答：圖 17-9 實測前五次推論的耗時是 163.6、43.8、36.2、32.0、32.3 毫秒，而穩態中位數是 32.83 毫秒。第一次慢了將近五倍。若把包含前幾次的所有量測取平均，會得到 41.78 毫秒，比真實的穩態值高估了約 27%。第一次慢的原因有：推論引擎要配置記憶體與工作緩衝區、為當前的張量形狀選擇最佳的運算核心（有些引擎會現場試跑幾種實作）、權重第一次被讀進快取、以及可能的即時編譯。正確的做法是先跑幾次不計時的暖機，再開始量測（程式 17-1 的 model(img) 那一行就是在做這件事）。但暖機在真實部署中並不是可以忽略的量測瑕疵，而是一個要設計的問題：機器人剛開機的第一個影格就會慢上好幾倍，若系統在開機後立刻開始移動，那一瞬間的反應能力與穩態完全不同。對策是在系統進入待命狀態時就先用一張假影像把模型跑熱，並且不要在暖機完成前允許機器人移動。同一個道理也適用於切換解析度或切換模型的時刻。

19. 編延遲預算時該用平均延遲、中位數、還是第 99 百分位?請依圖 17-12 的實測說明，並解釋為什麼對控制迴路而言這個選擇是安全問題。

    提示與參考解答：圖 17-12 實測同一個模型連續推論的延遲分布：中位數 35.3 毫秒、第 90 百分位 39.8、第 99 百分位 49.2、最大值 62。分布有明顯的長尾，來源是作業系統排程、其他行程搶佔 CPU、快取失效、以及影像內容本身的差異（物件多時後處理較久）。若以 40 毫秒為預算，實測有 21.5% 的影格超出——而中位數看起來明明還有餘裕。這就是為什麼不能用中位數或平均值編預算：它們描述「典型情況」，而即時系統的風險全在尾端。對串流播放來說，偶爾一格遲到只是輕微的不順;但對控制迴路而言，那一格就是控制訊號遲到，而機器人在那一瞬間仍然在以原速移動（推導 17.4.3 的反應距離）。所以安全攸關的系統應以 p99 甚至最大值編預算，並在這之上再留餘裕;同時必須定義「超時了怎麼辦」——沿用上一格的結果（並限制最多連續幾格）、降速、或停止，這就是 17.7 節失效即安全的起點。

20. 同一個系統，圖 17-11 說推論佔七成、圖 17-13 說推論只佔三成。這兩個數字並不矛盾。請說明差異從哪裡來，以及這對「該最佳化哪一段」的判斷有什麼意義。

    提示與參考解答：差別在量測邊界畫在哪裡。圖 17-11 量的是「視覺處理鏈」：影像解碼 8.75 毫秒（20.6%）、模型推論 29.73 毫秒（69.8%），合計 42.59 毫秒。在這個邊界內推論是絕對的主角，最佳化模型是對的。圖 17-13 量的是「完整的機器人反應鏈」：再加上相機曝光與傳輸、決策、通訊、以及致動器反應，總計約 145.9 毫秒，推論佔比掉到約三成，而致動器反應那一段比推論還久。意義有兩層。第一層是阿姆達爾定律（Amdahl's law）的實際後果：最佳化某一段所能帶來的整體改善，上限是那一段的佔比。即使把推論加速到零，圖 17-11 的鏈仍有 12.9 毫秒的地板，圖 17-13 的鏈仍有 100 毫秒以上。第二層更重要：先定義邊界再量測，否則會把力氣花錯地方。若你關心的是「機器人多久才會煞車」，那麼只盯著推論時間最佳化，很可能在佔三成的那一段拚命，卻對另外七成（例如換一顆延遲更低的相機、或縮短通訊的輪詢週期）完全沒動過。

21. 「批次處理可以提高吞吐量，代價是增加單張延遲」——這是教科書的標準說法，但圖 17-10 的實測顯示吞吐量反而下降。為什麼?那個標準說法在什麼條件下才成立?

    提示與參考解答：圖17-10 實測：batch=1 是18.5 FPS，batch=8 降到13.9 FPS。不是原地踏步，是倒退。標準說法成立的前提是「硬體還有閒置的平行處理能力」。GPU 有數千個運算單元，單張影像往往餵不滿，湊成一批可以讓這些單元同時工作，於是吞吐量上升。但本例的CPU 核心數少、而推論本身已經把核心用滿，沒有閒置的平行度可以利用;這時湊批次不但沒有好處，還多了更大的中間張量、更差的快取命中率與額外的記憶體搬運，於是整體變慢。所以批次處理的正確判準是：先看硬體利用率（GPU 的occupancy、CPU 的核心負載），只有在明顯有閒置時才值得試，而且要實測。順帶一提，即使在GPU 上批次有效，對機器人避障這類任務仍然不該用：湊滿一批的等待時間直接加在延遲上，而延遲才是這類任務的關鍵指標（17.4.1 節）。批次處理屬於離線影片分析的世界。

22. 若延遲不夠，可以不必每一影格都偵測——中間的影格沿用上一次的結果。這個做法的代價是什麼?請依圖 17-14 的實測量化，並說明如何設計一個安全的跳格策略。

    提示與參考解答：圖17-14 拿一段有移動物件的影片實測，把「沿用的舊框」與「同一影格上重新偵測得到的框」比較：每格偵測時IoU 是1.000、中心誤差0 像素;每2 格偵測降到0.927／5.5 像素;每5 格降到0.761／19.5 像素;每8 格只剩0.612／34.7 像素。所以代價是「位置資訊逐漸過期」，而且誤差隨跳格數近似線性成長。注意這個誤差與物件的移動速度、以及物件在畫面中的大小有關——快速接近的物件在影像中位移最大，而那正是最危險的情況。安全的跳格策略有三個要素。一、用追蹤器填補空檔：在不偵測的影格上用光流或KCF 等輕量追蹤器更新框的位置（第12 章），而不是完全沿用舊框;追蹤的成本遠低於偵測，卻能把誤差壓下來。二、把跳格數與速度掛鉤：機器人慢速時可以多跳，高速時必須每格偵測，或者反過來——高速時先降速再維持偵測頻率。三、設定沿用的上限：若連續N 格沒有新的偵測結果（例如偵測執行緒卡住），系統必須進入安全狀態，而不是無限期地相信一個越來越舊的框。

23. 現場的影像品質會以各種方式退化。若只能先解決一種，你會先解決哪一種?請依圖 17-17 的實測排序，並說明這個排序為什麼可能反直覺。

    提示與參考解答：圖17-17 對同一批保留集影像施加五種退化，量測mAP@0.5 的掉幅（百分點）：低光 −1.16、雜訊 −18.11、低解析 −20.85、JPEG 品質15 −23.42、運動模糊 −44.34。所以答案是先解決運動模糊，它的破壞力是低光的將近四十倍。這個排序反直覺的地方在於「低光」幾乎無害。原因是現代模型在訓練時已經見過大量曝光不足的影像，而且卷積網路主要依賴的是邊緣與紋理的相對關係，整體變暗並不會破壞這些關係（第8 章的亮度與對比調整也印證這點）;運動模糊卻直接把邊緣與紋理抹掉——它刪除的是模型賴以判斷的資訊本身。工程上的意義很具體：與其加裝補光燈，不如先把快門時間縮短（用全域快門、提高增益、或換一顆更靈敏的感測器，第7.6.3 節）、或是降低機器人與雲台的運動速度。同理，JPEG 壓縮品質與傳輸解析度這兩項是系統設計者可以直接控制的，卻常常為了省頻寬被調得太低——實測顯示它們的代價與雜訊相當，絕不是免費的。

24. 把推論搬進感測器裡，主機到底省下了什麼?請說明 IMX500 這條路徑與傳統路徑的差別，並解釋為什麼這個設計同時省了頻寬、功耗與隱私。

    提示與參考解答：傳統路徑是「感測器輸出影像 → 主機解碼與前處理 → 主機跑模型 → 得到結果」，這段期間主機的處理器幾乎做不了別的事。IMX500 的路徑是「感測器輸出影像，同時附上一包張量後設資料」，裡面就是模型的輸出——偵測

框、類別、分數;主機只要讀那包後設資料就好。三個好處都來自同一件事——影像沒有離開感測器。頻寬：要傳的只是幾個框的座標與分數，而不是一張影格。功耗：省掉傳輸，也省掉主機的解碼與推論，主機因此可以大部分時間休眠（圖17-4 的續航帳）。隱私：這是最有力的隱私保護設計——不是承諾不看，而是技術上沒有東西可看（17.1.1 節）。但要記住17.2.1 節的提醒：它買到的不是速度。感測器上的推論並不比主機快，它買到的是「主機的時間」與「不必搬影像」。所以它最適合的場景是主機還有別的正事要做、或者主機大部分時間應該在睡覺;若你的系統只有視覺這一件事而且主機閒著，好處就小得多——這又是一次「適合勝於最強」。

25. （綜合設計題）模型上線之後沒有標準答案，你要怎麼知道它開始失準了?請設計一套監控方案，列出不需要人工標註就能算的訊號、警示線的訂法、以及這套方案「看不見」什麼。

提示與參考解答：本題為開放設計題，評分重點在三層架構是否完整，以及是否誠實地指出侷限。參考方向——第一層自動監控，三個訊號。一、信心分數分布：記錄每張影像最高信心分數的移動平均，以及「幾乎沒偵測到東西」的影像比例。圖17-19 實測在影像變模糊的那一刻，前者從0.841 掉到0.443、後者從0% 升到34%，兩者都在零標註的條件下清楚指出異常。二、輸出統計量：每張影像的物件數、各類別比例、框的大小與位置分布。這些在穩定場域中應該穩定，而且與應用語意直接相關（產線每分鐘的工件數少了一成就該有人看）。三、系統指標：p99 延遲與超出預算的影格比例，它們能在硬體降頻或行程競爭「造成失效之前」就看出徵兆。警示線的訂法：上線初期先收集一段正常運作期的資料，算出平均與標準差，警示線訂在平均減去兩倍標準差，並留意季節性（戶外應用的日照、產線的班別）造成的正常波動。第二層定期抽樣複核：每週固定抽一批上線影像人工標註，算出真實的mAP 或召回率;這批資料必須是模型沒看過的（第16 章圖16-18），樣本數也要夠大才報得出有意義的數字（第14.5.3 節的標準誤）。第三層資料回饋：把複核出的錯誤案例標註後納入下一輪訓練，重訓後再走完整的驗收流程，並與舊版並行比較一段時間。看不見什麼（這是本題的關鍵）：上述訊號只能說「有事發生」，不能說「錯在哪裡」;而且有一整類失效它們完全偵測不到——模型自信滿滿地持續犯錯。第15 章的對抗脆弱、第16 章的捷徑學習都屬於這一類：信心高、輸出數量正常、延遲漂亮，但答案是錯的。所以自動監控永遠不能取代定期的人工抽樣複核。另外也要設計更新機制本身的安全性：更新失敗要能自動回復舊版、更新時機器人須先停止、新版要能與舊版並行比較後才全面切換。

參考資料與延伸閱讀

- Raspberry Pi 官方文件與 AI Camera 產品規格：https://www.raspberrypi.com(本章叫獸開講與 17.2.1 節之史料與規格來源)
- NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南：https://developer.nvidia.com/embedded-computing
- ROS 2 官方教學文件：https://docs.ros.org
- Howard, A. G., et al., MobileNets: Efficient CNNs for Mobile Vision Applications, arXiv, 2017.(第 15 章亦引用)
- Jacob, B., et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR, 2018.
- Ultralytics 部署與匯出文件：https://docs.ultralytics.com
- Sony IMX500 智慧視覺感光元件技術資料與模型轉換工具鏈說明。

- NVIDIA Jetson Orin 開發者文件與 TensorRT 最佳化指南：https://developer.nvidia.com/embedded-computing
- NVIDIA NanoOWL 專案（OWL-ViT 的 TensorRT 最佳化與樹狀提示）：https://github.com/NVIDIA-AI-IOT/nanoowl
- Minderer, M., et al., Simple Open-Vocabulary Object Detection with Vision Transformers (OWL-ViT), ECCV, 2022.
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)